IntraForge — Engineering Handbook

IntraForge Local Edition 1.5.7

Build. Store. Connect. Privately.

Contents

1. About this handbook
2. Chapter 1 — Architecture
3. Chapter 2 — The backend
4. Chapter 3 — The frontend
5. Chapter 4 — Security and permissions
6. Chapter 5 — Applications on IntraForge
7. Chapter 6 — The shop kit: build, install, upgrade, operate
8. Chapter 7 — The other repositories
9. Chapter 8 — Working on IntraForge: setup, tests, release
10. Chapter 9 — Known limits, open items and how to scale
11. Reference A — the API
12. Reference B — the data model
13. Reference C — the modules

About this handbook

This handbook is for the software engineer who takes over, extends or scales IntraForge. It explains how the whole system is built, why it is built that way, and where every piece of code lives. Read Chapters 1–4 first: they are the rules every change must keep. The last three parts are **generated from the code itself** and list every API endpoint (Reference A), every database model (Reference B) and every source file (Reference C).

If you want to…	Read
Understand the system in an hour	Chapter 1 (architecture), Chapter 4 (security)
Change the backend	Chapter 2, then the app's section in Reference C
Change a screen	Chapter 3
Add or change a business application	Chapter 5
Build a kit, install or upgrade a shop	Chapter 6
Work on Pay, the wallet app or the standalone Spare Parts	Chapter 7
Set up your machine, test and release	Chapter 8
Know what is not finished or not proven	Chapter 9

Where the truth is. When this handbook and a document under `docs/` disagree, the document wins for its topic: architecture decisions in `docs/architecture/adr/`, permissions in `docs/security/PERMISSIONS.md`, threats in `docs/security/THREAT_MODEL.md`, operations in `docs/operations/`. `CLAUDE.md` at the repository root holds the standing engineering rules. Regenerate the three reference parts after any change (Chapter 8).

What IntraForge is

IntraForge ("Build. Store. Connect. Privately.") is a private business platform that runs on the customer's own PC or server, with no internet needed for normal work. It combines:

- **File storage** — buckets, folders, versions, share links, a trash, malware scanning.
- **A no-code database builder** — real PostgreSQL tables, rows, CSV import, analytics, read-only connections to outside PostgreSQL, MySQL and SQL Server databases.
- **An application platform** — business applications (Spare Parts / Inventory & Sales, Pharmacy, Repair Shop, Rentals, CRM, HR…) defined as data, installed per organization, run by one generic engine.
- **Service applications** — separately built programs (IntraForge Pay, the Spare Parts shop app, the Pharmacy app) that use IntraForge's sign-in, storage and audit.
- **Shop features** — the Movie Shop with its customer portal and media agent, hosted websites, shop names on the LAN, licensing.
- **People and safety** — organizations, teams, roles and capabilities, two-step sign-in and passkeys, an audit log nobody can edit, encrypted backups that are test-restored automatically.

The repositories

Repository (folder)	What it holds	Language
`intra-cloude` (this one; branch `integration/1.5.0`)	IntraForge itself: backend, frontend, shop kit, Control Center, setup, media agent, docs	Python, TypeScript, PowerShell, C#
`intraforge-pay` (and its `-portal`, `-wallet` working copies)	IntraForge Pay: the shop server, cashier and office pages, the double-entry ledger, the Android wallet	Python (FastAPI), SQL, Kotlin
`spare-part-shop`	The standalone Spare Parts shop application (catalogue, quotations, invoices, labels)	TypeScript (Next.js), SQLite
`licensing` (private: never in a repo, kit or shop PC)	The vendor's licence-signing key and serial-number registry	—

Versions at the time of writing

IntraForge Local Edition **1.5.7** (kit `1.5.7-586a4bbe8`) runs at the pilot shop.

- Backend: Python 3.13, Django 5.2, Django REST Framework 3.18, Celery 5.6, psycopg 3, boto3.
- Data: PostgreSQL 18 (two servers), Valkey 9, MinIO.
- Frontend: Node 24, Next.js 16.4, React 19.2, TanStack Query 5, Tailwind CSS 4.
- Proxy: Caddy 2. Shop names: CoreDNS 1.15.
- Windows side: Docker Desktop with WSL 2; C# 5 on .NET Framework 4.x for the Control Center, Setup and the media agent, compiled with the compiler that ships with Windows.

Chapter 1 — Architecture

The big picture



Everything runs as Docker containers on one machine, on a private Docker network **with no route to the internet**. Only the proxy publishes ports, and by default only on the loopback address (`127.0.0.1` and `::1`); opening it to the shop network is a deliberate step (Control Center > Shop network, `Set-IntraForgeNetwork.ps1`).

The containers

Service	Image	Role
`proxy`	`caddy:2-alpine`	The only published port: 8443 (IntraForge), 8445 (hosted sites), 8446/8080 internal helpers. Refuses `/api/v1/internal/` from outside.
`frontend`	`intraforge-shop-frontend`	The Next.js screens, rendered in the browser against the API.
`backend`	`intraforge-shop-backend`	Django + DRF: every rule, permission check and write.
`worker`	same image	Celery worker: imports, exports, restores, backups, webhook deliveries, provisioning.
`beat`	same image	Celery Beat: nightly backups, weekly restore tests, health checks every 5 minutes.

Service	Image	Role
`runner`	same image (`python -m app_runner`)	Starts hosted Node.js apps in locked containers; the backend talks to it by token, never to Docker itself.
`postgres-control`	`postgres:18`	The control plane: users, organizations, roles, grants, app definitions, audit, backups, licence.
`postgres-tenant`	`postgres:18`	The data plane: one schema per organization for tables people build and installed apps' records; one schema per service-app instance.
`valkey`	`valkey/valkey:9-alpine`	Cache (shared rate-limit counters) and the Celery queue.
`object-storage`	`minio/minio`	Every file: uploads, versions, covers, backups' file parts, hosted-site files.
`pay-1` … `pay-3`	`intraforge-shop-pay`	IntraForge Pay deployments (one per Pay instance).
`spareparts`, `pharmacy`	own images	The standalone shop applications, served under `/apps/...`.
`names` (profile `names`)	`coredns/coredns`	Answers the shop's names (`myshop.lan`, `movies.lan`) on the shop network.

The two planes (ADR-0001, ADR-0005)

Django is the **control plane**: it *describes, authorizes and audits* access to data. It never stores large files and is never the store for business data. The **data plane** is PostgreSQL (tenant database) and object storage.

- **postgres-control** holds the catalog — who exists, what they may do, which resources exist and where.
- **postgres-tenant** holds business data. Each organization gets its own PostgreSQL schema, so one organization's tables are physically separate from another's, not rows filtered by an `organization_id`.
- **Object storage** holds files; Django keeps only their metadata (`FileObject`, `FileVersion`).

`config/db_routers.py` keeps Django's own models on the control database; tenant tables are reached through explicit, quoted SQL (`databases/rows.py`, `app_platform/records.py`) on the tenant connection.

How a request travels

1. The browser calls `https://localhost:8443/api/v1/...`. Caddy terminates HTTPS and forwards `/api/*` to `backend:8000`.
2. Django's middleware runs in order: security headers, a request id (`system.middleware.RequestIDMiddleware`), sessions, CSRF, authentication, **`PasswordChangeMiddleware`** (a person whose password was reset may do nothing until they choose a new one), **`LicenceMiddleware`** (read-only without a licence), **`ApiRequestLogMiddleware`** (records requests made with an application's token).
3. DRF authenticates the caller: a session cookie for people, `Authorization: Bearer pdc_sk_…` for applications.
4. The view is thin: it parses input with a serializer and calls a **service function**.

5. The service resolves the resource **inside the caller's organization** (a foreign id answers 404, not 403, so it does not reveal that it exists), checks the **capability** with `permissions.services.has_permission`, does the work in a transaction and writes an **audit event**.
6. Slow work is handed to Celery (`worker`) through Valkey; the screen polls the job.

The rules the design never breaks

1. **Control plane and data plane stay separate** (ADR-0001).
2. **Tenant isolation is a backend invariant.** Every tenant resource carries its organization; every request is checked on the server. New resource types ship only with tests proving Organization A cannot reach B's data by id.
3. **No unreviewed dynamic SQL.** Identifiers from users are validated (`databases/identifiers.py`) and quoted with `psycopg.sql.Identifier`; DDL runs in a transaction, after a permission check, with an audit event.
4. **Capabilities, never role names** (ADR-0008). Code asks "does this person hold `inventory.sell` here?", never "is this an admin?".
5. **Schema per organization** (ADR-0005).
6. **Boring technology first.** Docker Compose, not Kubernetes (ADR-0006). No new datastore, identity provider or orchestrator without a new ADR.
7. **Local-first.** Nothing needs the internet. Internet exposure (hosted sites through a Cloudflare Tunnel) is off by default and only the Super Administrator can switch it on.

Architecture decision records

ADR	Decision
0001	Control plane (Django) separate from data plane (PostgreSQL + object storage)
0002	Django + DRF in small bounded apps
0003	Django-native authentication first, OIDC-ready
0004	S3-compatible object storage, MinIO locally
0005	Schema-per-organization tenant isolation
0006	Docker Compose first, Kubernetes deferred
0007	TypeScript + React, Next.js
0008	Capability-based authorization
0009	External databases: connected (read-only) vs imported modes
0010 / 0011	Celery for background jobs; Valkey instead of Redis (licence)
0012 / 0013	Windows deployment (WSL2 appliance route) and Docker Engine in the rootfs — the earlier MSI route
0014 / 0015	App Platform: immutable template versions, independent instances; resumable runtime provisioning
0016	Tenant operation journal and quantity ledgers (the database decides stock)
0017	Business-operation authority over stock (sell / receive / adjust / transfer)
0018	Service applications on IntraForge storage, identity and audit
0019	Passkeys (WebAuthn)

ADR	Decision
0021	Shop names on the LAN (CoreDNS)
0022	Movie Shop and the media agent
0023	Licensing: trial, serial number, activation code
0024	MySQL and SQL Server connectors

Hosted websites and the app runner are described in code as "ADR-0019" and "ADR-0020" decisions, but no ADR file with those decisions exists (0019 is passkeys). See Chapter 9.

Chapter 2 — The backend

Layout

```
apps/backend/
  config/            settings (base, dev, prod, test), urls.py, api_urls.py, celery.py,
db_routers.py
  <app>/             one bounded Django app per domain (below)
    models.py        control-plane models
    services.py      the business logic: permission checks, transactions, audit events
    views.py         thin DRF views: parse, call a service, shape the answer
    serializers.py   input validation and output shape
    urls.py          the app's routes (mounted under /api/v1/ by config/api_urls.py)
    tasks.py         Celery tasks (when the app has background work)
    migrations/      schema history — never edited after release
    tests/           the app's tests, against real PostgreSQL and MinIO
  app_runner/        the separate process that runs hosted Node.js apps in locked containers
  requirements/      base.txt, prod.txt, dev.txt — exact pins
```

Settings come from environment variables only (`config/settings/base.py`); the kit writes them into `.env` with fresh random secrets per shop. `config.settings.test` is used by `pytest` (see `pytest.ini`).

The apps

App	Responsibility	Key modules
`accounts`	People: sign-in, sessions, CSRF, TOTP two-step, passkeys, recovery codes, password change and reset	`services.py` , `totp.py` , `passkeys.py` , `middleware.py`
`organizations`	Organizations, members, invitations, teams	`services.py`
`permissions`	The capability catalog, roles, role assignments, resource grants; **the one authorization service**	`catalog.py` , `services.py`
`org_units`	Organizational hierarchy (branches, departments) and unit-scoped access	`services.py`
`workspaces`	Workspaces and projects inside an organization	`views.py`
`storage`	Buckets, folders, files, versions, trash, share links, cleanup, upload limits, ClamAV scanning	`services.py` , `backends.py` , `scanning.py`
`databases`	The visual schema builder (real tenant tables), rows, exports; connected external PostgreSQL/MySQL/SQL Server	`ddl.py` , `identifiers.py` , `rows.py` , `connectors.py`
`imports`	CSV inspection (encoding, delimiter, types) and background bulk insert	`inspect.py` , `tasks.py`
`analytics`	A fixed registry of statistics (numpy/scipy), table profiling, dashboards	`operations.py` , `profiling.py`
`sharing`	Share grants, compiled into resource grants	`services.py`

App	Responsibility	Key modules
`applications`	API clients: Application, ServiceAccount, bearer credentials, request log	`authentication.py`, `middleware.py`
`oauth_provider`	IntraForge as an OAuth/OIDC sign-in provider for other apps, with a consent screen	`services.py`
`environments`	Development / Staging / Production, variables, encrypted secrets, signed webhooks	`services.py`, `crypto.py`
`app_platform`	The application platform (Chapter 5)	many; see Reference C
`service_apps`	Separately deployed apps (Pay, Spare Parts, Pharmacy): registry, provisioning, internal API, Pay bridge	`services.py`, `internal_views.py`, `pay_bridge.py`
`movie_shop`	Movie Shop catalogue, counter sales, media-agent API, customer portal, ads	`services.py`, `sales.py`, `portal.py`
`sites`	Hosted static websites, the app runner client, the internet gate (Cloudflare Tunnel)	`services.py`, `serve.py`, `internet.py`
`licensing`	Trial, serial number, activation code; read-only without a licence	`codes.py`, `middleware.py`
`catalogue_storage`	Storage for the spare-parts catalogue integration	`services.py`
`audit`	The immutable audit trail	`services.py` (`record`)
`exports`	Portable `.icp` export/import of a whole organization, idempotent restore	`builder.py`, `restorer.py`, `container.py`
`system`	`/healthz`, `/readyz`, `/metrics`, health history and alerts, backups and restore tests, diagnostics, throttling	`backups.py`, `health.py`, `redaction.py`

The service-layer pattern

Business logic lives in services, never in views or serializers. A service:

1. **Finds the resource inside the caller's organization** (`get_member_bucket`, `get_member_table`, …). A resource of another organization raises `Http404` — the caller cannot tell whether it exists.
2. **Checks the capability** through `permissions.services.has_permission(user, code, organization_id, resource)`. Effective permission = a role held in that organization (or platform-wide) **or** a non-expired resource grant on that exact resource.
3. **Does the work in a transaction.** DDL on the tenant database runs first; if writing the catalog row then fails, a compensating DDL undoes it (`databases.services._write_catalog`).
4. **Records an audit event** with `audit.services.record(actor=…, organization_id=…, action=…, result=…)`, including a `denied` event when the permission check refuses.

A shortened real example (`databases/services.py`):

```
def create_tenant_database(*, actor, project, name, request_id=""):
    org_id = project.organization_id
    _require(actor, "database.create", org_id, action="database.create",
             resource_type="tenant_database", resource_id="", request_id=request_id)  # audits
"denied" too
    tenant_db = TenantDatabase(project=project, name=name, created_by=actor)
    tenant_db.schema_name = f"db_{tenant_db.id.hex}"
    _execute_tenant_ddl(sql.SQL("CREATE SCHEMA
{s}").format(s=sql.Identifier(tenant_db.schema_name)))
    _write_catalog(tenant_db.save,
                   compensating_ddl=sql.SQL("DROP SCHEMA {s}
CASCADE").format(s=sql.Identifier(tenant_db.schema_name)))
    audit.record(actor=actor, organization_id=org_id, action="database.create",
                 resource_type="tenant_database", resource_id=tenant_db.id, context={"name":
name})
    return tenant_db
```

The view only translates: serializer in, service call, `Response` out, and maps service exceptions to status codes.

Dynamic SQL

Tables, columns and constraints that people create are real PostgreSQL objects. The rules (`databases/identifiers.py`, `ddl.py`, `rows.py`):

- identifiers are validated against a strict pattern and a reserved-word list before use;
- every identifier is quoted with `psycopg.sql.Identifier`, every value is a bound parameter — never string formatting;
- DDL runs in a transaction after the permission check and writes an audit event;
- reads are capped (rows and bytes) so one request cannot exhaust the server.

Background jobs (Celery)

`worker` runs tasks from Valkey; `beat` schedules the periodic ones (`config/celery.py`). Long tasks set `acks_late=True` with a visibility timeout longer than the task, so a worker that dies mid-task has the task delivered again — and every such task is written to be safe to run twice (imports keep a checkpoint; exports and restores carry idempotency keys and operation-derived ids; webhooks de-duplicate by event id). Schedules: backups nightly (02:00–02:50), restore tests weekly, health checks every 5 minutes, webhook retries with back-off.

Migrations

- Every model change ships with a migration; never edit a released migration.
- **Rollback safety:** the kit can roll an upgrade back to the previous images while the database keeps the newer schema. A new non-null column therefore needs a database default (`db_default=`), so the older code can still insert rows. Never drop or rename a column in the same release that stops using it.
- The shop upgrade runs `migrate` itself; on the test copy run `docker compose exec -T backend python manage.py migrate`.

Errors

`system/exceptions.py` turns errors into `{"error": {"code": …, "message": …, "request_id": …}}`. A stack trace is never sent to the browser; it is logged with the request id, which the screen can show under "View technical details".

Rate limits

`DEFAULT_THROTTLE_RATES`: anonymous 30/min, signed-in 300/min, **auth 10/min** (sign-in, register, MFA — intentional; never loosen it, space tests instead), import 20/min per organization. Counters live in Valkey (`CACHES`), so all gunicorn workers share one budget.

Management commands worth knowing

Command	What it does
`seed_permissions`	Loads the capability catalog and system roles from `permissions/catalog.py`
`bootstrap_super_administrator`	Creates the installation's first Super Administrator
`passkey_reset`	Removes a person's passkeys (support)
`seed_reference_app`, `seed_spare_parts_reference_app`	Installs a reference application into an organization
`upgrade_operation_journals`	Additive upgrade of existing app runtimes' journals

Chapter 3 — The frontend

Layout

```
apps/frontend/
  src/app/                  Next.js App Router: one folder per route
    (app)/                  every signed-in screen, wrapped by layout.tsx -> AppShell (sidebar,
header, palette)
    login/ register/ welcome/ oauth-consent/ share/[token]/ portal/   public screens
    globals.css             design tokens (colour variables, light and dark), Tailwind v4 theme
  src/components/           shared components; ui.tsx is the component kit
    business/               the built-in Spare Parts / Pharmacy screens (BusinessApp, SellStock,
ReceiveStock, Labels...)
    movies/  portal/        Movie Shop owner screens and the customer portal
  src/lib/                  API client, types, query layer, auth, helpers (each with its .test.ts)
  e2e/                      Playwright browser tests
```

The app is **client-rendered**: pages are React client components that call the API in the browser through the same Caddy origin, so there is no CORS and no server-side data access in Next.js.

Talking to the API — `src/lib/api.ts`

- `api.get / post / patch / delete` call `/api/v1/...` with `credentials: "include"` (the session cookie).
- Before the first write, `ensureCsrfCookie()` calls `GET /auth/csrf/`; every mutating call sends `X-CSRFToken`.
- Errors become `ApiError` with `status`, the backend's `code` and `requestId`. `ErrorBanner` shows a friendly message and offers "View technical details".
- The response types (`User`, `Organization`, `Bucket`, `DBTable` …) live in the same file — keep them in step with the serializers (Reference A lists the endpoints).

Data on screen — `src/lib/query.tsx`, `queries.ts`

TanStack Query caches page data in the tab's memory: a page seen before shows at once and refreshes in the background (30 s stale time). A 403/404 is never retried. The cache is cleared whenever the signed-in person changes (`auth-context.tsx`), so one person's data never shows to another.

Signing in — `src/lib/auth-context.tsx`

`useAuth()` exposes `user`, `login` (which may answer `mfaRequired`), `verifyMfa`, `signInWithPasskey`, `register`, `logout`, `refresh`. Passkeys use WebAuthn through `src/lib/passkeys.ts`. After a password reset the backend refuses everything except changing the password; `PasswordForm` handles that screen.

The component kit — `src/components/ui.tsx`

Use these instead of raw elements so every screen looks and behaves the same: `Button` (primary, secondary, danger, ghost), `LinkButton`, `Input`, `Textarea`, `Select`, `Checkbox`, `Label`, `Card`, `Badge`, `Spinner`, `PageLoading`, `EmptyState`, `ErrorBanner`, `CopyButton`, `SecretReveal` (a secret shown once), `PageHeader`, `Table` / `THead` / `Th` / `Td` / `TRow`, `Modal` (focus trap via `use-dialog-a11y.ts`), `StatCard`. Confirmations go through `ConfirmProvider`.

Design rules

- **Colours are tokens** (`--color-brand-600`, `--color-navy`, …) defined in `globals.css` and redefined for dark mode under `html[data-theme="dark"]`. `ThemeToggle` and `lib/theme.ts` store the choice (system / light / dark) in the browser. Never hard-code a hex colour in a component. The brand is described in `DESIGN.md`.
- **Icons are `lucide-react` components, never emoji** (emoji render differently across fonts and platforms).
- Accessible by default: labels on every input, visible focus rings, `Modal` traps focus, motion respects `prefers-reduced-motion` (`use-reduced-motion.ts`).
- The screen hiding a button is a courtesy; the server's capability check is the lock. Always handle a 403 with a clear "you don't have permission" message.

Routes

The signed-in routes (inside `(app)/`) include: `/dashboard`, `/apps` and `/apps/[instanceId]/…` (the business app: products, inventory, receive, sales, customers, suppliers, warehouses, part numbers), `/orgs/[orgId]/…` (storage, databases, audit, teams, units, sites, app templates, the developer area), `/buckets/[bucketId]`, `/tables/[tableId]` (with `/import` and `/analytics`), `/tenant-databases/[dbId]`, `/connected-databases/[id]`, `/dashboards/[id]`, `/environments/[id]`, `/applications/[id]`, `/service-apps/[appId]/open|settings`, `/movies` and `/movies/[shopId]`, `/account`, `/backups`, `/health`, `/license`, `/internet`. Public: `/login`, `/register`, `/welcome`, `/oauth-consent`, `/share/[token]`, `/portal` and `/portal/[shopId]` (the Movie Shop customer portal, phone layout).

Adding a screen

1. Create `src/app/(app)/<route>/page.tsx` as a client component; use `PageHeader` and the kit.
2. Add the response type to `lib/api.ts` and a query hook (or `useQuery` inline) keyed by the resource id.
3. Handle loading (`PageLoading`), empty (`EmptyState`), error (`ErrorBanner`) and 403 states.
4. Add it to the sidebar (`AppShell.tsx`) or the command palette (`CommandPalette.tsx`) if people need to find it.
5. Add a Vitest test for any logic in `lib/`, and a Playwright spec for a whole workflow.

Tests

- `npm test` — Vitest unit tests (`*.test.ts(x)` next to the code).
- `npm run e2e` — Playwright against a running stack (`e2e/`: auth, tenant isolation, the Spare Parts cashier, flagship and accessibility journeys, app platform).
- `npm run lint`, `npx tsc --noEmit` (after `npm run build`), `npm audit --omit=dev --audit-level=high`.

Chapter 4 — Security and permissions

The detailed, authoritative documents are `docs/security/PERMISSIONS.md` (every capability and the rule for each endpoint) and `docs/security/THREAT_MODEL.md` (trust boundaries, every gap found and how it was closed). This chapter is the working summary.

Who is calling

Caller	How they prove it	Where
A person	Email + password → session cookie; optional TOTP second step; passkeys (WebAuthn); recovery codes	`accounts`
An application	`Authorization: Bearer pdc_sk_…` → its ServiceAccount's identity user	`applications/authentication.py`
A service application (Pay, Spare Parts, Pharmacy)	Its deployment's service token on the **internal** API only	`service_apps/authentication.py`
The media agent	That Movie Shop's own agent credential	`movie_shop/views.py`
The app runner	A shared runner token, fixed operations only	`sites/runner.py`, `app_runner/`
A portal customer	A portal account for one shop (phone number + password)	`movie_shop/portal.py`

Person details:

- **TOTP** (RFC 6238) is implemented directly in `accounts/totp.py` with the standard library; secrets are encrypted at rest with Fernet (`CREDENTIAL_ENCRYPTION_KEY`).
- **Recovery codes** are stored hashed; each works once.
- **Password reset by an administrator** sets `must_change_password`; `PasswordChangeMiddleware` then refuses every API call except "who am I", changing the password and signing out.
- New administrative role grants can require MFA in internet-gateway mode.
- Sign-in endpoints are throttled at **10 per minute** on purpose.

What they may do — capabilities (ADR-0008)

A **capability** is a permission string. Code never branches on a role name; it calls:

```
from permissions.services import has_permission
has_permission(user, "inventory.sell", organization_id=org.id, resource=("app_instance", instance.id))
```

Effective permission = a **role** held in that organization (or platform-wide, for the Super Administrator) whose capability set includes the code, **or** a non-expired **ResourceGrant** for that exact capability on that exact resource. Sharing (`ShareGrant`) compiles down to ResourceGrants — there is no second enforcement path.

Capability families (all in `permissions/catalog.py`, the single source of truth that `seed_permissions` loads):

Family	Examples
Platform	`system.admin`, `backup.manage`, `users.manage`, `permissions.manage`, `audit.read`, `sites.internet.manage`
Workspaces, storage	`workspace.manage`, `storage.read/write/delete/share/manage`, `sharing.manage`
Databases, data	`database.create/read/write/delete/schema.manage`, `dataset.import/export/analyze`, `connection.manage`
Integration	`application.create/read/credentials.manage`, `environment.read/manage/secrets.manage/production.manage`, `export.manage`
App platform	`app_template.read/manage/publish`, `app_instance.read/manage/schema.manage/workflow.approve`, `org_unit.*`
Stock (ADR-0017)	`inventory.sell`, `inventory.receive`, `inventory.adjust`, `inventory.transfer` — separate, none implies another
Service apps	`ifpay.`, *`spareparts.`*, `pharmacy.*`
Shop features	`movies.read/sell/manage`, `sites.read/manage`

System roles are named bundles: Super Administrator (everything, platform-wide), Organization Administrator, Storage Administrator, Database Administrator, Developer, Editor, Viewer, Auditor, Guest, Service Account, and the app roles (Spare Parts Manager/Sales/Inventory/Viewer, Pay Manager/Supervisor/Cashier, Pharmacy Manager/Pharmacist/Cashier/ Accountant). **Adding a capability:** add it to `catalog.py` and to `PERMISSIONS.md`, re-run `seed_permissions`, check it in the service, and test both the allowed and the denied path.

Tenant isolation

- Every tenant resource carries its organization. Lookups go through `get_member_*` helpers that filter by the caller's memberships first; another organization's id answers **404**.
- Business data sits in a per-organization PostgreSQL schema; a service application's data in its own schema with its own login role (`service_apps/provisioning.py`).
- **Environment isolation:** an Environment-scoped application credential can never reach a database or bucket bound to another Environment (enforced in `databases`, `storage`, `imports`).
- **Every new tenant resource type needs IDOR tests** — Organization A substituting B's ids must fail — before it ships.

The audit log

`audit.services.record(...)` writes who, what, which resource, the result (`success`, `denied`, `error`) and the request id. The model refuses updates and deletes. Denied attempts are recorded too. Organization admins and auditors read it at `/orgs/[orgId]/audit`.

Secrets and encryption

What	How
MFA secrets, connected-database passwords, environment secrets	Fernet, key from `CREDENTIAL_ENCRYPTION_KEY`
Environment secrets, application credentials	Shown exactly once; stored encrypted or hashed
Backups and `.icp` exports	AES-256-GCM with an Argon2id-derived key (`exports/container.py`)
Webhooks	HMAC-signed bodies, at-least-once delivery, receivers de-duplicate by event id
Licences	Ed25519-signed activation codes; the private key never leaves the vendor (`licensing/codes.py`)
Diagnostics	Secrets redacted before anything is written (`system/redaction.py`)

The kit generates fresh random secrets per shop into `.env`. Never commit `.env`, keys or passwords; never print a secret into a log.

Network and input defences

- Only the proxy is published, on loopback by default; the Docker network has no internet route.
- `/api/v1/internal/` (service applications) is refused by the proxy from outside.
- **SSRF guard:** connected databases and webhook destinations resolve the host, refuse private/loopback/link-local addresses unless allowed, and pin the resolved address (`databases/connectors.py` , `app_platform/outbound_http.py`).
- Uploads have size limits; with ClamAV enabled, a file that cannot be scanned is quarantined (fail-closed).
- Hosted-site zips are fully validated (names, types, sizes, counts) before anything is stored; sites are served on a separate origin (port 8445) with cookies stripped.
- The internet gate is off by default; only the Super Administrator, with password and two-step sign-in, can publish a hosted site through a Cloudflare Tunnel. IntraForge itself is never published.

Rules for every change

- Default deny: a new endpoint checks a capability before doing anything.
- Never trust the client to name an operation, an organization or an amount the server can work out itself.
- Stock arithmetic happens only in PostgreSQL (ADR-0016); never in the browser or a view.
- Run `npm audit --omit=dev --audit-level=high` and `pip-audit` for every release; pins never update themselves.
- When a security-sensitive choice is unclear, stop, write the open question down (doc or ADR) and choose the established conservative pattern.

Chapter 5 — Applications on IntraForge

IntraForge runs business applications in three different ways. Know which one you are touching:

Kind	Example	Code lives in	Data lives in
Built-in app (App Platform) — defined as data, run by one generic engine	Spare Parts / Inventory & Sales, Pharmacy, Repair Shop, Rentals	`app_platform/` + `frontend/src/components/business/`	The organization's tenant schema
Service application — a separate program with its own image	IntraForge Pay, the Spare Parts shop app, the Pharmacy app	Its own repository; `service_apps/` is IntraForge's side	Its own schema in the tenant database (+ IntraForge storage)
Feature app — a normal IntraForge app with its own models	Movie Shop, hosted websites	`movie_shop/`, `sites/`	The control database (metadata) + object storage; films stay on the shop's drives

5.1 The App Platform (ADR-0014 – ADR-0017)

An application is a **definition**, not copied code.

1. **Template** (`AppTemplate`, `AppTemplateVersion`) — models, fields, relationships, constraints, workflows, labels. The format is strict, bounded and non-executable (`definitions.py`). A published version is **immutable**.
2. **Instance** (`AppInstance`) — one organization's installation of a version (`instances.py`).
3. **Runtime** — real tables in the organization's tenant schema, built by **provisioning** (`runtime_plan.py` plans, `provisioning.py` reserves and replays, `runtime_build.py` builds inside transactions). Provisioning is resumable: a crash part-way is finished on the next attempt, never duplicated.
4. **Records** — one generic CRUD engine for every instance (`records.py`, `record_views.py`), with relationships, attachments (`attachments.py`), unit-scoped access (`unit_scope.py`) and workflow transitions with optional approval (`workflows.py`).
5. **Schema evolution** — additive-only live changes to a populated runtime (`schema_evolution.py`).
6. **Upgrades** — a three-way diff between the install snapshot, the live configuration and the newer version: adds what is safe, never silently overwrites the customer's own changes (`upgrades.py`). Definitions get deterministic ids (`built_in_apps.stabilize_ids`) so a diff compares like with like.
7. **Webhooks** — record and workflow events delivered as HMAC-signed POSTs, **at least once**, with durable retry (`webhooks.py`, `delivery_transport.py`, `outbound_http.py`).

The gallery and release status

`app_platform/built_in_apps.py` is the customer catalog. `install()` does the whole template → instance → runtime sequence in one call through the same services (so every capability check still applies). Status is real:

App	Status	Shown to customers
Spare Parts / Inventory & Sales	QUALIFIED	Yes
Pharmacy	PARTIALLY QUALIFIED	Only with `SHOW_UNQUALIFIED_BUILT_IN_APPS`

App	Status	Shown to customers
HR, School, Repair Shop, Rentals	EXPERIMENTAL	Only with the flag, clearly labelled
CRM, Procurement, Assets, Projects, Documents, Warehouses	DEFERRED	Never

Promote an app to QUALIFIED only with evidence (main workflow in the normal UI, relationships, constraints, permissions, restart, upgrade, backup and restore) and record where it lives in the entry's `evidence`.

Stock: the ledger and operation authority

- **The database decides stock** (ADR-0016). Stock is a quantity ledger (`quantity_ledger.py`): a balance changes only by a movement, and PostgreSQL triggers enforce that balance = sum of movements and never below zero. No code in the browser, a view or a service does stock arithmetic.
- **Authority belongs to the operation** (ADR-0017, `inventory_authority.py`): `inventory.sell`, `.receive`, `.adjust`, `.transfer` are separate, default-deny capabilities. A cashier can sell but not receive.
- Selling and receiving go through `operations/sell` and `operations/receive` (`inventory_operations.py`): one atomic transaction (`transactions.py`), an **Idempotency-Key** is required (a double click cannot sell twice), human sessions only. A movement written through the generic record API needs `inventory.adjust`. Never add a way for the client to name the operation.
- Every record change is journalled in the tenant transaction and published to the control plane afterwards, replayably (`operation_journal.py`).

Adding a new built-in application

1. Write a pure builder in `app_platform/reference_templates/<key>.py` using `_helpers.py` (copy `repair_shop.py` or `rentals.py` for a small one; `spare_parts.py` if it needs stock).
2. Add a `BuiltInApp` entry in `built_in_apps.py` with status EXPERIMENTAL.
3. If it needs custom screens, add them under `frontend/src/components/business/`; otherwise the generic record screens serve it.
4. Test: the builder validates; install provisions; records CRUD; permissions deny the wrong role; IDOR across organizations; upgrade from the previous version.

5.2 Service applications (ADR-0018)

A service application is a separately built program that runs as its own container and uses IntraForge for identity, storage, settings and audit.

- **Types** (`service_apps/types.py`) declare each app's capabilities, the one needed to open it, its settings schema (validated by IntraForge and again by the app) and the tables counted in the removal confirmation.
- **Deployments** (`deployments.py`, `SERVICE_APP_DEPLOYMENTS`) are the containers that exist in this installation: `pay-1` … `pay-3`, `spareparts`, `pharmacy`. Each gets a service token and a database password in `.env` — only the values it needs, never the whole file.
- **Provisioning** (`provisioning.py`) creates one PostgreSQL login role per deployment and one schema per installed instance in the tenant database.

- **Open** (`views.py`, `sessions.py`): a person with the open capability clicks **Apps > Open**; IntraForge creates a short-lived browser session for that app and redirects to `/apps/<deployment>/`.
- **The internal API** (`internal_views.py`, `/api/v1/internal/`): reachable only on the internal network, authenticated by the deployment's token, scoped to that deployment's own instance — there is no way to name another instance. It serves the person's identity and capabilities, settings, file storage and audit writes.
- **The Pay bridge** (`pay_bridge.py`, `app_pay.py`): other apps (the Movie Shop, a hosted Node.js app with its `IF_API_TOKEN`) ask the organization's Pay to take a payment; IntraForge picks the Pay and passes the result back.

5.3 The Movie Shop (ADR-0022)

- **Catalogue** (`models.py`, `scanning.py`): only metadata lives in IntraForge. The films stay on the shop's drives; covers are ordinary files in the shop's `movie-covers` bucket. Every folder in a library becomes a category.
- **The media agent** (`scripts/shop-kit/media-agent/MediaAgent.cs`) is a small Windows program on the shop PC. It reports drives, scans library folders and copies sold titles onto the customer's USB drive. IntraForge checks every value it sends (`scanning.py`) — nothing is trusted because it came from our own agent.
- **Counter sales** (`sales.py`): a cart for one customer, paid through IntraForge Pay or cash, then copied by the agent; each title's copy progress is tracked; a failed copy can be retried.
- **The customer portal** (`portal.py`, `portal_views.py`, frontend `components/portal/`): customers sign up on the shop Wi-Fi (`https://movies.lan:8443` through the shop-names service), wait for approval, browse and order, and collect points (welcome, referral, birthday, every Nth title free). Ads (`ads.py`) show on the portal.
- Capabilities: `movies.read`, `movies.sell`, `movies.manage`.

5.4 Hosted websites and the app runner

- **Static sites** (`sites/`): an organization uploads a finished website as a zip (validated by `archive.py`); each upload is an immutable `SiteVersion` stored in object storage; one version is active. Sites are served on port 8445, a separate origin, by path (`https://<pc>:8445/<site>/`) or by shop name (`https://myshop.lan:8445/`).
- **Server apps**: the `runner` container starts a hosted Node.js app in its own locked container (non-root, read-only, no capabilities, memory/CPU limits, no internet). The backend never talks to Docker; it calls the runner's fixed operations with a token (`sites/runner.py`).
- **A worked example:** `examples/hosted-apps/internet-cafe/` is a complete server app — an internet café website that keeps its data in an IntraForge database (read/write grant only), takes payments through the Pay bridge (`POST /api/v1/pay/payment-requests/` with its `IF_API_TOKEN`) and has its own tests. Copy its `lib/intraforge.js` and `lib/store.js` patterns for a new hosted app.
- **Internet access** (`sites/internet.py`): off by default; the Super Administrator can publish a chosen site through a Cloudflare Tunnel. Only hosted sites are ever published — never IntraForge, its API or a service app.

5.5 Licensing (ADR-0023)

A new installation has a 30-day trial. Then it needs a **serial number** and an **activation code** made for that PC's code (`licensing/codes.py`, Ed25519-signed by the vendor tool `scripts/licensing/intraforge-license.py`). Without a valid licence, `LicenceMiddleware` makes IntraForge **read-only**: anything that would change data is refused except signing in and out, consent screens, activating the licence and the internal service API. Data is never lost. The private key and the serial registry live only with the vendor — never in a repository, kit or shop PC.

Chapter 6 — The shop kit: build, install, upgrade, operate

The shop runs IntraForge from the **Docker-runtime kit** in `scripts/shop-kit/`, delivered as one file, `IntraForge-Setup-<version>.exe`. (The older WSL2/MSI route in `installer/` is not qualified and not used.)

What is in the kit

Part	Source	Notes
Container images (saved as `.tar`)	built from committed source	backend, frontend, Pay, Spare Parts, Pharmacy + pinned postgres, valkey, MinIO, caddy, coredns
`docker-compose.yml`	`template/docker-compose.yml`	`@TAG@` / `@MINIO_TAG@` filled in at build time; `pull_policy: never` everywhere
PowerShell scripts	`template/*.ps1`	install, start, stop, status, backup, restore, upgrade, uninstall, network
Control Center	`control-center/*.cs`	the shop owner's Windows window (status, start/stop, backups, restore, shop network, diagnostics)
Media agent	`media-agent/MediaAgent.cs`	the Movie Shop's file copier
Setup	`setup-stub/*.cs`	the self-extracting installer window
Manifest	generated	every component with its SHA-256; no secrets, no demo data

Building a kit (on the build machine, online, Docker Desktop running)

```
KIT_VERSION=1.5.7 CARRY_APPS_FROM=1.4.0-dbf44f593 MINIO_TAG=RELEASE.2025-09-07T16-13-09Z \
PAY_IMAGE=intraforge-shop-pay:1.5.6-counters PAY_SOURCE="<pay repo and commit>" \
SPAREPARTS_IMAGE=intraforge-shop-spareparts:labels PHARMACY_IMAGE=intraforge-shop-pharmacy:labels
\
SPAREPARTS_SOURCE="<spare-part-shop commit>" \
bash scripts/shop-kit/build-shop-kit.sh <output-folder>
bash scripts/shop-kit/build-setup-exe.sh <output-folder>
```

- The kit is built from **committed source only** (`git archive HEAD`); `scripts/shop-kit` must have no uncommitted changes or the build stops.
- The result is `IntraForge-Local-<version>-<commit>.zip` + `.sha256`, then `IntraForge-Setup-<version>-<commit>.exe` (Setup = stub + zip + optional Docker Desktop and WSL installers, each with its SHA-256 in a trailer).
- The C# programs are compiled with `csc.exe` from .NET Framework 4.x, which every Windows 10/11 has — no SDK needed.

Installing on a shop PC

`Setup-IntraForge.ps1` (run by the Setup window) does, in order: check the PC (Windows version, memory, disk, virtualization) → switch on the Windows features Docker needs and restart (`Prepare-WindowsFeatures.ps1`) → install WSL and Docker Desktop after checking their signatures (`Install-Prerequisites.ps1`) → `Install-IntraForgeShop.ps1`:

1. Check the kit is complete and unchanged (SHA-256).
2. Create the data folder (`C:\IntraForgeData`) and **fresh random secret keys** in `.env` (a copy goes to the data folder; the owner must keep a copy off the PC).
3. Load the images from the kit (no download).
4. Start the databases and storage; run migrations and `seed_permissions`.
5. Start IntraForge and wait until `/readyz` answers.
6. Register start at Windows sign-in (`HKCU\...\Run`), the daily backup task (19:00), desktop shortcuts and the Windows "Apps" uninstall entry.

Upgrading

`IntraForge-Setup-<new>.exe /quiet "/dir=C:\IntraForge"` (or double-click) runs `Upgrade-IntraForgeShop.ps1`:

1. **Safety backup of everything** with the installed version.
2. Keep the previous program files in `.previous`.
3. Stop the previous version (data stays), install the new files, add any settings the new version needs to `.env`.
4. Load the new images, run migrations and update the applications.
5. Start the new version and wait for `/readyz`.
6. **If anything fails: automatic rollback** — the previous files and keys are put back and the previous version is started with the same data. This is why new columns need `db_default` (Chapter 2).

The media agent is stopped to replace its program and restarted afterwards, from the install folder (started from the staging folder it would hold that folder open and block the next upgrade).

Day-to-day scripts

Script	What it does
`Start-IntraForgeShop.ps1`	Starts IntraForge and opens the browser; runs at sign-in (`-Auto` also takes a backup if the last is older than 20 h)
`Stop-IntraForgeShop.ps1`	Stops IntraForge; all data kept
`Status-IntraForgeShop.ps1`	Health summary, also saved to a file the project team can read
`Backup-IntraForgeShop.ps1`	Full backup (company data, business records, files) into the backups folder; `-Prune` keeps a bounded history
`Restore-IntraForgeShop.ps1`	**Destructive**: puts the shop back to a saved backup (needs the original keys file)

Script	What it does
`Repair-IntraForgeBackups.ps1`	Fixes "pg_dump: Permission denied" backup folders
`Set-IntraForgeNetwork.ps1`	Lets other PCs in the shop open IntraForge (binds to the LAN address, firewall rule, shop names) or makes it this-PC-only again
`Uninstall-IntraForgeShop.ps1`	Removes IntraForge in one of three ways (keep data, keep backups only, remove everything)
`IntraForgeShop.Common.ps1`	Shared helpers, dot-sourced by the others (Windows PowerShell 5.1)

Shop network and names (ADR-0021)

By default IntraForge answers only on `127.0.0.1` / `::1`. **Control Center > Shop network** (or `Set-IntraForgeNetwork.ps1`) binds the proxy to the PC's LAN address, opens the firewall for the private network and starts the `names` service (CoreDNS, compose profile `names`), which answers the shop's names (`myshop.lan`, `movies.lan`) with the PC's address and passes everything else to the router. Point the router's DNS at the PC (or set it per device) so phones find `movies.lan`.

When the PC changes network (for example to a phone hotspot), the saved LAN address no longer exists and Docker refuses to bind port 53 on it. The start and upgrade scripts then use the `without-names` profile so IntraForge still starts; **Shop network > Update the address** fixes it. (In Windows PowerShell, setting `$env:COMPOSE_PROFILES=''` *deletes* the variable, so an explicit profile name is used instead of an empty one.)

Backups on a shop PC

- In the app: nightly backups of the control database, the tenant database, object storage and configuration (02:00–02:50), all encrypted; a weekly restore test into a throw-away target; the Backups page shows the result.
- From the kit: a daily backup at 19:00 (a shop PC is often off at 02:00), "Backup to USB", a safety backup before every upgrade.
- A full recovery needs all four parts from the same day **and the original `.env`**.

The Control Center (`control-center/`)

A native Windows program (C# 5, WinForms, GDI+ drawing in the DESIGN.md style — `Ui.cs`). It reads the installed kit's `.env` for the port and data folder, shows status, starts and stops, backs up and restores, manages the shop network, the media agent and the licence code, and collects diagnostics. `INTRAFORGE_CC_RENDER_UI=<folder>` renders every screen to PNG files without changing anything (used for the guide screenshots).

The Setup window (`setup-stub/`)

An 880×560 borderless window (`Ui/SetupForm.cs`, `Ui/Pages.cs`) over a self-extracting payload (`Payload.cs`: reads the trailer, checks each part's SHA-256, unpacks). Modes: interactive, `/quiet`, `/check`, `/extractonly`, `/uipreview=<screen>`. A fresh staging folder is used if an old one is still held open by another process.

Do not

- Do not read or copy a real shop's `.env` or keys file; diagnostics redact secrets for a reason.
- Do not put the licensing private key or the serial registry in a kit, repository or shop PC.
- Do not claim a Windows restart, LAN access or code signing was tested unless the shop PC's status file shows it.

Chapter 7 — The other repositories

7.1 IntraForge Pay (`intraforge-pay`)

Simulated value — no cash value. Pay is the shop's QR wallet: customers hold IF Credits, the cashier shows a QR code, the customer confirms in the wallet. It runs inside IntraForge as a service application (ADR-0018): image `intraforge-shop-pay`, containers `pay-1` … `pay-3`, served at `/apps/pay-N/`. Working branches: `main`, `feature/counters` (counters, migration 0006), `fix/android-gradlew-lf`.

Layout

Path	What it is
`ifpay/web/app.py`	FastAPI app; pages `/shop` (cashier), `/wallet` (customer), `/admin` (office), `/login`
`ifpay/web/api_shop.py`, `api_wallet.py`, `api_admin.py`, `api_auth.py`, `api_internal.py`	The API by audience
`ifpay/ledger.py`	**The double-entry journal.** Every movement of credits is a balanced journal entry
`ifpay/money.py`	Amounts are integers of hundredths (2500 = 25.00). Never floats
`ifpay/checkout.py`, `qr.py`, `keys.py`	QR payment requests, HMAC-signed codes; the signing key lives in Pay's own schema
`ifpay/receipts.py`, `refunds.py`, `rewards.py`	One immutable receipt per purchase; maker/checker refunds; cashback
`ifpay/allocations.py`	Maker/checker allocation of credits (a manager asks, a supervisor approves)
`ifpay/counters.py`	Named counters (tills); each payment records its counter; "today" in the shop's time zone
`ifpay/audit.py`	Append-only audit (the table rejects UPDATE/DELETE/TRUNCATE)
`ifpay/platform.py`, `runtime.py`	IntraForge's internal API: waits to be installed, opens the provisioned schema
`ifpay/documents.py`	Receipts and documents in the shop's IntraForge bucket
`ifpay/security.py`, `tls.py`	Sessions, CSRF, capabilities; the "connect to this shop" pairing code for phones
`ifpay/migrations/000N_*.sql`	Forward-only SQL migrations with database-enforced rules (`db.py` runs them)
`tests/`, `tests_money/`	Tests against real PostgreSQL, and live tests against a real IntraForge
`android/`	The **IF Credits Wallet** Android app

Rules

- The ledger must always balance; the database enforces it. Never update or delete a journal row — correct with a new entry.
- Money is integer hundredths end to end (database, Python, JSON).
- Refunds and allocations need two different people (maker/checker).
- Pay capabilities come from IntraForge roles: Pay Cashier (`ifpay.checkout`), Pay Supervisor, Pay Manager.

The Android wallet (`android/`, Kotlin, Jetpack Compose)

`MainActivity.kt`, `Screens.kt` (UI), `WalletViewModel.kt` (state), `PilotApi.kt` (HTTP to the shop's Pay), `Shops.kt` (paired shops and their pinned certificate — `ShopTrustTest.kt`), `Theme.kt`. Build with the Gradle wrapper (`./gradlew assembleDebug`; the wrapper must have LF line endings). The last debug build is `kits/IFPayWallet-debug-70680a3.apk`; it has not yet been tested on a real phone.

7.2 The Spare Parts / Pharmacy shop application (`spare-part-shop`)

A standalone, offline-first Next.js application for a spare-parts shop: reference catalogue (manufacturer part numbers, fitment, drawings), products with photos and prices, stock across warehouses and racks, customers and suppliers, the chain **enquiry** → **quotation** → **order** → **invoice** → **payment**, PDFs, barcode/QR labels and reports. The same engine runs the **Pharmacy** edition (`APP_EDITION=pharmacy`, `src/server/edition.ts`). Branch: `feature/labels-shop-name`.

Path	What it is
`src/app/`	Next.js routes (`(app)/` screens, `api/` handlers)
`src/server/domain/`	The business rules: products, inventory, purchasing, sales, documents, reports, permissions
`src/server/providers/`	Interfaces (identity, audit, files, configuration, data, unit of work) the domain depends on
`src/server/standalone/`	Providers for running alone (SQLite via `node:sqlite`, local files)
`src/server/platform/`	Providers for running **inside IntraForge**: identity and capabilities from IntraForge, files in IntraForge storage, audit through an outbox to IntraForge's log, configuration from IntraForge, Pay
`src/server/db/`	SQLite connection and migrations; `pg-migrations/` + `postgres.ts` for the IntraForge-provisioned schema
`src/server/catalogue/`	Catalogue package import (CSV)
`src/server/documents/`, `labels/`	PDF documents and labels (the shop's name, product, captioned part number)
`src/ui/`	The screen kit

Inside IntraForge it is built by `scripts/shop-kit/build-spareparts-image.sh` from a **committed** revision (`git archive`, never `.env` or local data) and served at `/apps/spare-parts/` and `/apps/pharmacy/`. IntraForge's roles map to its permissions in `platform/capabilities.ts` (pinned by a test).

Do not confuse it with IntraForge's **built-in** Spare Parts / Inventory & Sales (Chapter 5.1), which is defined as an App Platform template and is the qualified application.

7.3 Licensing tool (vendor only)

`scripts/licensing/intraforge-license.py` makes serial numbers and activation codes with the vendor's private key, using the same rules as `licensing/codes.py`. The private key (`licensing/private-key.pem`) and the serial registry stay in the private `licensing` folder on the vendor's machine: never in a repository, a kit or a shop PC.

Chapter 8 — Working on IntraForge: setup, tests, release

Setting up a development machine

Needs: Git, Docker Desktop (with WSL 2 on Windows), Node 24, Python 3.13 (for the docs tools and linters).

1. Copy `.env.example` to `.env` and fill in the secrets (database passwords, `SECRET_KEY`, object-storage root credentials, `CREDENTIAL_ENCRYPTION_KEY`). Never commit `.env`.
2. `docker compose build`
3. `docker compose up -d postgres-control postgres-tenant valkey object-storage` and wait for the health checks.
4. `docker compose run --rm backend python manage.py migrate`
5. `docker compose run --rm backend python manage.py seed_permissions`
6. `docker compose run --rm backend python manage.py bootstrap_super_administrator you@example.com`
7. `docker compose up -d backend worker beat frontend proxy`
8. Open `https://localhost:8443/` (accept the local certificate, or trust Caddy's root CA).

For a disposable copy for tests, drills and demos use the qualification stack (`scripts/qualification/compose.yaml`, port 8444). A full shop-like test copy can be installed from a kit into its own folder (the project's own was `test-1.5.0`, port 9543), with test-only logins kept beside it.

Tests

What	Command	Notes
Backend lint	`ruff check .` and `ruff format` (line length 110) in `apps/backend`	Format only the files you touched
Backend types	`mypy .`	
Backend tests	`pytest` (from the repo root; `pytest.ini` points at `apps/backend` and `tests`)	Real PostgreSQL, MinIO and Valkey, never mocks; `--reuse-db` by default, `--create-db` after migrations change
Dependency audit	`pip-audit -r requirements/base.txt -r requirements/prod.txt`	Every release
Frontend unit	`npm test` (Vitest)	
Frontend lint/types	`npm run lint`, `npm run build`, `npx tsc --noEmit`	`tsc` after `build` (it reads `.next/types`)
Frontend audit	`npm audit --omit=dev --audit-level=high`	Every release
Browser journeys	`npm run e2e` (Playwright) against a running stack	Auth, tenant isolation, Spare Parts cashier/flagship/accessibility, app platform

What	Command	Notes
Pay	`pytest` in the Pay repo	Real PostgreSQL; live tests need an IntraForge
Kit scripts	`scripts/shop-kit/tests/` (Pester) and a VM test (`vm-test/`)	

CI (`.github/workflows/ci.yml`) runs the backend lint, types, audit and tests; the frontend audit, lint, unit tests, build and type check; a full Docker stack with Playwright; and image builds.

Rate limit in tests: sign-in is limited to 10 per minute on purpose. Space or isolate tests that sign in; never raise the limit.

Adding a feature — the checklist

1. **Read first:** `CLAUDE.md` , this handbook's Chapters 1–4, and the ADRs for the area.
2. **Decide where it lives:** an existing bounded app, or a new app if it is a new domain. A new datastore, identity mechanism or orchestration change needs a new ADR first.
3. **Model + migration** (with `db_default` on new non-null columns, for rollback safety).
4. **Service function** with the organization-scoped lookup, the capability check (and a `denied` audit on refusal), a transaction and a success audit event. New capability → `permissions/catalog.py` + `PERMISSIONS.md` .
5. **Thin view + serializer + URL.**
6. **Tests:** the happy path, the denied path, input validation, and **cross-organization IDOR** for any new tenant resource. For background work: a test that running the task twice is safe.
7. **Screen** (Chapter 3) with loading, empty, error and 403 states; a Vitest test for any logic.
8. **Docs** in the same change: the relevant doc under `docs/` , the user guide if people see it, and regenerate the handbook's reference parts:

```
docker compose exec -T backend python manage.py shell < docs/guides/tools/gen_reference.py >
ref.md
python docs/guides/tools/gen_modules.py
python docs/guides/tools/build_guides.py
```

(`gen_reference.py` prints both reference parts separated by a line `=====MODELS=====` ; put the first part in `engineering/90-reference-api.md` and the second in `91-reference-models.md` .)

Releasing

1. Run every test and both dependency audits; fix or record anything high.
2. Commit everything — the kit is built from `HEAD` only.
3. Build the images for any changed service app (Pay, Spare Parts) from their committed revisions.
4. Build the kit and the Setup exe (Chapter 6), with a new `KIT_VERSION` .
5. Install it on a clean test copy and upgrade an existing test copy; check `/readyz` , a sale, a backup and a restore test.
6. Upgrade the pilot shop with the Setup exe; confirm the status file and the Backups page afterwards.
7. Record the release in `replica/build-log.md` and the checkpoint docs.

Branches and commits

- Work on a branch (current integration branch: `integration/1.5.0`); never push to `master` / `main` and never force-push. Changes reach `master` through pull requests the owner merges.
- Small commits, one concern each, in the conventional style: `feat(storage): …`, `fix(kit): …`, `docs(guides): …`.
- Ask before changing a separate repository (Pay, Spare Parts).

Regenerating the guides and screenshots

Tool	What it does
`docs/guides/tools/capture-screens.cjs`	Playwright: signs in to a test copy and saves the web screenshots in `docs/guides/screenshots/web/`
`INTRAFORGE_CC_RENDER_UI=<folder>` + the Control Center	Renders every Control Center screen to PNG
`docs/guides/tools/gen_reference.py`, `gen_modules.py`	The generated reference parts of this handbook
`docs/guides/tools/build_guides.py`	Markdown → HTML → PDF (Microsoft Edge headless, no downloads)
`docs/guides/tools/build_ai_pack.py`	Assembles the AnythingLLM knowledge pack

Chapter 9 — Known limits, open items and how to scale

Honest status

Area	Status
Spare Parts / Inventory & Sales on the Docker runtime	**Qualified** (search, receive, sell, documents, files, restricted users, backup and recovery, offline, upgrade, accessibility, small-shop scale)
Pilot shop	Runs 1.5.7 on a real Windows PC; upgrades with automatic rollback have worked there
One-file Windows Setup	Tested on a clean Windows 11 VM and on the pilot PC
Pharmacy	Partially qualified
HR, School, Repair Shop, Rentals	Experimental
MySQL / SQL Server connectors	Tested against real MySQL 8.4 and SQL Server 2022 containers; not yet used at a customer
IF Credits Wallet (Android)	Debug build only; **not tested on a real phone**
Not qualified / not proven	The MSI/WSL2 route; code signing (no certificate); an external security scan; LAN access from a second PC (guide written: `docs/pilot/SECOND_PC_TEST_GUIDE.md`); commercial distribution (the MinIO AGPL and Docker Desktop licences need decisions first)

IntraForge Pay credits are **simulated**. Real money would need a licensed payment partner, a legal review and a new design for settlement — not a configuration change.

Open items worth knowing

- **Missing ADR files.** Hosted websites and the app runner cite "ADR-0019" and "ADR-0020" decisions in their code, but `docs/architecture/adr/` has 0019 = passkeys and no 0020. Write those two decisions down (as 0025/0026) and fix the references.
- **Undocumented code.** Reference C marks with a dash every file that has no description yet (many `models.py`, `views.py`, `serializers.py`, `urls.py`). The API reference shows an empty description where a view has no docstring. Adding one-line docstrings improves both references and the AI pack automatically.
- `docs/development/CONTRIBUTING.md` is still the Phase 0 placeholder; Chapter 8 of this handbook is the current practice.
- **Configuration restore** is deliberately manual (the other three backup kinds restore automatically).
- `.icp` **exports** do not include App Platform data; use full backups for that.
- **Languages.** The interface is English only. Arabic, Amharic and Tigrinya for Spare Parts were estimated at about three working days for screens plus one for printed documents, and need a decision on fonts and on per-person vs shop-wide language.

- **Reports, returns, quotations, stock transfers and tax compliance** are outside the qualified built-in Spare Parts (the standalone Spare Parts app has quotations and invoices).

How to scale

IntraForge was designed for one shop on one PC, but its boundaries were chosen so it can grow without a rewrite.

Need	What to do	What stays the same
More users in one shop	Raise gunicorn workers and Celery concurrency; give PostgreSQL more memory	Everything else — throttles already share Valkey
A bigger server	Move the same compose file to a Linux server (ADR-0012's customer-managed host)	The kit's images and data volumes
Several shops, one company	One installation, one organization with **org units** (branches); unit-scoped grants limit each branch's staff	Capabilities, audit, the ledger
Several companies	Separate organizations on one installation — schema-per-organization isolation already holds	IDOR tests for every resource
Heavy files	Point object storage at a larger S3-compatible store (ADR-0004); files never pass through Django's database	The `storage` service API
High availability	Managed PostgreSQL with replicas, several backend containers behind the proxy, Valkey with persistence — then consider an orchestrator (needs a new ADR; ADR-0006 defers Kubernetes until a concrete need)	The service layer and data model
Other programs connecting	Applications with scoped credentials, Environments, webhooks, OAuth sign-in (`docs/EXTERNAL_APP_API_CONTRACT.md`)	Default deny
A new business type	A new App Platform template (Chapter 5.1) — data, not code	The generic runtime, ledger, permissions

Do not scale by weakening a rule: no shared tables filtered only by `organization_id`, no stock arithmetic outside PostgreSQL, no role-name checks, no internet dependency for normal work, no loosened rate limits.

Where to read more

- `docs/HOW_INTRAFORGE_WORKS.md` — a plain-language tour.
- `docs/architecture/ARCHITECTURE.md`, `docs/APP_PLATFORM_ARCHITECTURE.md`, `docs/architecture/adr/`.
- `docs/security/PERMISSIONS.md`, `docs/security/THREAT_MODEL.md`.
- `docs/operations/` — backups, restore, update policy, diagnostics, SBOM, third-party licences.
- `docs/implementation/CURRENT_STATE.md`, `docs/LOCAL_EDITION_V1_FINAL_QUALIFICATION.md` — evidence and checkpoints.
- `docs/guide/USER_GUIDE.md` and the Training Guide PDF — how people use it.

Reference A — the API

Every endpoint, generated from the URL configuration and the views' own descriptions. Paths under `api/v1/` need a signed-in session (or an application's bearer credential where the view allows it); each view checks its capability itself (see Chapter 4). `<uuid:...>` is an id.

accounts

Path	Methods	View	What it does
`/api/v1/auth/csrf/`	GET	`CSRFView`	`GET /auth/csrf/` — a browser SPA hits this once on load so Django actually sets the `csrftoken` cookie (it's otherwise only set as a side effect of `get_token()` running during some other request, e.g. the first authenticated mutation, which is too late — the
`/api/v1/auth/login/`	POST	`LoginView`	
`/api/v1/auth/logout/`	POST	`LogoutView`	
`/api/v1/auth/me/`	GET	`MeView`	
`/api/v1/auth/mfa/confirm/`	POST	`MFAConfirmView`	
`/api/v1/auth/mfa/disable/`	POST	`MFADisableView`	
`/api/v1/auth/mfa/enroll/`	POST	`MFAEnrollView`	
`/api/v1/auth/mfa/recovery-codes/`	GET, POST	`MFARecoveryCodesView`	GET: how many recovery codes are left. POST (a current 6-digit code): 10 new ones, the old ones gone.
`/api/v1/auth/mfa/verify/`	POST	`MFALoginVerifyView`	
`/api/v1/auth/passkeys/`	GET	`PasskeyListView`	`GET /auth/passkeys/` -- the signed-in user's own passkeys (never anyone else's).
`/api/v1/auth/passkeys/<uuid:passkey_id>/remove/`	POST	`PasskeyRemoveView`	
`/api/v1/auth/passkeys/check/options/`	POST	`PasskeyCheckOptionsView`	Signed in: a fresh check with one of the user's own passkeys. Signed out: passkey sign-in.
`/api/v1/auth/passkeys/check/verify/`	POST	`PasskeyCheckVerifyView`	

Path	Methods	View	What it does
`/api/v1/auth/passkeys/only/`	POST	`PasskeyOnlyView`	`POST /auth/passkeys/only/` with `{"on": true, "password": ...}`, or `{"on": false}` after a fresh passkey check in this session.
`/api/v1/auth/passkeys/register/options/`	POST	`PasskeyRegisterOptionsView`	
`/api/v1/auth/passkeys/register/verify/`	POST	`PasskeyRegisterVerifyView`	
`/api/v1/auth/password/`	POST	`PasswordChangeView`	The person changes their own password (their current one, then a new one). Their session stays signed in.
`/api/v1/auth/register/`	POST	`RegisterView`	

analytics

Path	Methods	View	What it does
`/api/v1/dashboards/<uuid:dashboard_id>/`	GET, PATCH, DELETE	`DashboardDetailView`	
`/api/v1/dashboards/<uuid:dashboard_id>/render/`	GET	`DashboardRenderView`	
`/api/v1/tables/<uuid:table_id>/analyze/`	POST	`TableAnalyzeView`	
`/api/v1/tables/<uuid:table_id>/profile/`	GET	`TableProfileView`	
`/api/v1/tenant-databases/<uuid:tenant_database_id>/dashboards/`	GET, POST	`DashboardListCreateView`	

app_platform

Path	Methods	View	What it does
`/api/v1/app-constraints/<uuid:object_id>/`	GET, PATCH	`ConstraintDetail`	
`/api/v1/app-fields/<uuid:object_id>/`	GET, PATCH	`FieldDetail`	
`/api/v1/app-fields/<uuid:object_id>/indexed/`	POST	`FieldIndexedView`	
`/api/v1/app-fields/<uuid:object_id>/unique/`	POST	`FieldUniqueView`	Dedicated action, human-session-only (schema mutation stays out of this first bearer-token slice -- see FoundationView's own docstring): retrofits a UNIQUE constraint onto an existing field, safely against an already-provisioned, possibly populated runtime.

Path	Methods	View	What it does
`/api/v1/app-instances/<uuid:object_id>/`	GET, PATCH	`InstanceDetail`	
`/api/v1/app-instances/<uuid:object_id>/built-in/retry/`	POST	`BuiltInAppRetry`	Finish preparing a built-in application whose setup did not complete. Idempotent.
`/api/v1/app-instances/<uuid:object_id>/models/`	GET, POST	`ModelList`	
`/api/v1/app-instances/<uuid:object_id>/operations/receive/`	POST	`ReceiveView`	
`/api/v1/app-instances/<uuid:object_id>/operations/sell/`	POST	`SellView`	
`/api/v1/app-instances/<uuid:object_id>/quantity-ledgers/`	GET, POST	`QuantityLedgerList`	
`/api/v1/app-instances/<uuid:object_id>/relationships/`	GET, POST	`RelationshipList`	
`/api/v1/app-instances/<uuid:object_id>/runtime-plan/`	GET	`RuntimePlan`	
`/api/v1/app-instances/<uuid:object_id>/runtime/`	GET, POST	`Runtime`	
`/api/v1/app-instances/<uuid:object_id>/schema/`	GET	`InstanceSchema`	An instance's whole schema -- every model with its fields, and every relationship -- in ONE read-only request and a constant number of queries. A client that needs the schema (the customer screens resolve model/field/relationship ids from it) would otherwise i
`/api/v1/app-instances/<uuid:object_id>/transactions/`	POST	`TransactionView`	Several record operations committed as one unit (transactions.py). Human session only: no `service_account_methods`, so an external bearer- token client keeps using ordinary single-record CRUD. Authorization is per operation, inside transactions.run -> records
`/api/v1/app-instances/<uuid:object_id>/upgrade/<uuid:version_id>/`	GET, POST	`UpgradeView`	GET previews a three-way diff against a target template version without changing anything; POST applies it (additions and safe label syncs only -- see upgrades.py for exactly what "safe" means and why). Human-session-only: an upgrade is schema administration,
`/api/v1/app-instances/<uuid:object_id>/upgrades/`	GET	`UpgradeHistoryList`	

Path	Methods	View	What it does
`/api/v1/app-models/<uuid:object_id>/`	GET, PATCH	`DefinitionDetail`	
`/api/v1/app-models/<uuid:object_id>/constraints/`	GET, POST	`ConstraintList`	
`/api/v1/app-models/<uuid:object_id>/fields/`	GET, POST	`FieldList`	
`/api/v1/app-models/<uuid:object_id>/records/`	GET, POST	`RecordListCreateView`	
`/api/v1/app-models/<uuid:object_id>/records/<uuid:record_id>/`	GET, PATCH, DELETE	`RecordDetailView`	
`/api/v1/app-models/<uuid:object_id>/records/<uuid:record_id>/attachments/`	GET, POST	`AttachmentListCreateView`	
`/api/v1/app-models/<uuid:object_id>/records/<uuid:record_id>/attachments/<uuid:attachment_id>/`	DELETE	`AttachmentDetailView`	
`/api/v1/app-models/<uuid:object_id>/records/<uuid:record_id>/attachments/<uuid:attachment_id>/download/`	GET	`AttachmentDownloadView`	
`/api/v1/app-models/<uuid:object_id>/records/<uuid:record_id>/transitions/<uuid:transition_id>/apply/`	POST	`ApplyTransitionView`	Applying a transition is fundamentally a record write -- open to a scoped bearer token the same way record CRUD already is (Phase 7), unlike every workflow *authoring* endpoint above, which stays human-only. All real authorization (database.write, plus app_ins
`/api/v1/app-models/<uuid:object_id>/unit-grants/`	GET, POST	`UnitScopeGrantList`	Phase 7 Step 3 ("unit scope"): an administrative sharing action, never something a bearer token needs to perform on its own behalf -- human-session-only, no service_account_methods opt-in.
`/api/v1/app-models/<uuid:object_id>/workflows/`	GET, POST	`WorkflowList`	
`/api/v1/app-relationships/<uuid:object_id>/`	GET, PATCH	`RelationshipDetail`	

Path	Methods	View	What it does
`/api/v1/app-template-versions/<uuid:object_id>/`	GET	`VersionDetail`	
`/api/v1/app-templates/<uuid:object_id>/`	GET, PATCH	`TemplateDetail`	
`/api/v1/app-templates/<uuid:object_id>/versions/`	GET, POST	`VersionList`	
`/api/v1/app-transitions/<uuid:object_id>/`	GET, PATCH	`TransitionDetail`	
`/api/v1/app-unit-grants/<uuid:object_id>/`	DELETE	`UnitScopeGrantDetail`	
`/api/v1/app-workflow-states/<uuid:object_id>/`	GET, PATCH	`StateDetail`	
`/api/v1/app-workflows/<uuid:object_id>/`	GET, PATCH	`WorkflowDetail`	
`/api/v1/app-workflows/<uuid:object_id>/states/`	GET, POST	`StateList`	
`/api/v1/app-workflows/<uuid:object_id>/transitions/`	GET, POST	`TransitionList`	
`/api/v1/built-in-apps/`	GET	`BuiltInAppList`	What this deployment offers a customer -- QUALIFIED applications only, unless SHOW_UNQUALIFIED_BUILT_IN_APPS is on. With `?project=<id>` each entry also lists the instances of it in that project the caller may read, and their runtime state. Human sessions only
`/api/v1/organizations/<uuid:organization_id>/app-templates/`	GET, POST	`TemplateList`	
`/api/v1/projects/<uuid:project_id>/app-instances/`	GET, POST	`InstanceList`	
`/api/v1/projects/<uuid:project_id>/built-in-apps/<str:key>/install/`	POST	`BuiltInAppInstall`	Install one built-in application into a project the caller is a member of. Authorization is unchanged: every underlying service checks its own capability.

applications

Path	Methods	View	What it does
`/api/v1/applications/<uuid:application_id>/`	GET	`ApplicationDetailView`	
`/api/v1/applications/<uuid:application_id>/credentials/`	GET, POST	`ApplicationCredentialListCreateView`	
`/api/v1/applications/<uuid:application_id>/credentials/<uuid:credential_id>/revoke/`	POST	`ApplicationCredentialRevokeView`	
`/api/v1/applications/<uuid:application_id>/credentials/<uuid:credential_id>/rotate/`	POST	`ApplicationCredentialRotateView`	
`/api/v1/applications/<uuid:application_id>/requests/`	GET	`ApplicationRequestLogView`	The application's recent requests (application.read): newest first, paginated; ?errors=1 shows only the ones that did not succeed (status 400 and above).
`/api/v1/applications/<uuid:application_id>/resource-grants/`	GET, POST	`ApplicationResourceGrantListCreateView`	
`/api/v1/applications/<uuid:application_id>/resource-grants/<uuid:grant_id>/`	DELETE	`ApplicationResourceGrantDetailView`	Revokes one of the application's own grants (e.g. taking IntraForge Pay away from a hosted app).
`/api/v1/organizations/<uuid:organization_id>/applications/`	GET, POST	`ApplicationListCreateView`	

audit

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/audit/`	GET	`AuditEventListView`	Real filtering/pagination over the organization's full audit history — the previous implementation returned only the most recent 200 events with no way to reach anything older or narrow by action/ actor/date, which made it useless for an actual incident invest

catalogue_storage

Path	Methods	View	What it does
`/api/v1/catalogue/diagrams/<uuid:diagram_id>/`	GET	`DiagramDetailView`	`GET /api/v1/catalogue/diagrams/{id}/` -- the diagram plus its stored figure snapshot (every part number drawn on it), served from this database alone so it works with no external access.
`/api/v1/catalogue/diagrams/<uuid:diagram_id>/image/`	GET	`DiagramImageView`	`GET /api/v1/catalogue/diagrams/{id}/image/` — only ever serves a LOCALLY_STORED diagram's actual bytes (streamed through the backend, same reasoning as storage.views.FileDownloadView: the object storage endpoint is an internal Docker DNS name). An EXTERNAL_RE

Path	Methods	View	What it does
`/api/v1/catalogue/diagrams/import/`	POST	`DiagramFigureImportView`	`POST /api/v1/catalogue/diagrams/import/` -- persists one whole figure (see services.import_figure). Same gate as OEMPartImportView: only an ApplicationCredential bearer token may write; metadata only, never image bytes.
`/api/v1/catalogue/oem-parts/`	GET	`OEMPartSearchView`	`GET /api/v1/catalogue/oem-parts/?q=<part number, any format>` — the one search this integration needs: exact match on the normalized part number (see services.search_part). Reachable by either a human session or this deployment's spare-parts ApplicationCreden
`/api/v1/catalogue/oem-parts/<uuid:part_id>/`	GET	`OEMPartDetailView`	
`/api/v1/catalogue/oem-parts/import/`	POST	`OEMPartImportView`	`POST /api/v1/catalogue/oem-parts/import/` — lets an external, bearer-authenticated caller (the spare-parts backend, resolving a live provider lookup on a cache miss) persist exactly one part+ diagram+link, through the identical idempotent path the CSV importe

databases

Path	Methods	View	What it does
`/api/v1/connected-databases/<uuid:connected_database_id>/`	GET, DELETE	`ConnectedDatabaseDetailView`	
`/api/v1/connected-databases/<uuid:connected_database_id>/schema/`	GET	`ConnectedDatabaseSchemaView`	
`/api/v1/connected-databases/<uuid:connected_database_id>/tables/<str:table_name>/rows/`	GET	`ConnectedDatabaseTableRowsView`	
`/api/v1/connected-databases/<uuid:connected_database_id>/test/`	POST	`ConnectedDatabaseTestView`	
`/api/v1/organizations/<uuid:organization_id>/tenant-databases/`	GET	`OrganizationTenantDatabaseListView`	Every database in an organization the caller may read (database.read, role-wide or a grant on that database, the same check as each project's list), with how many tables it has.
`/api/v1/projects/<uuid:project_id>/connected-databases/`	GET, POST	`ConnectedDatabaseListCreateView`	
`/api/v1/projects/<uuid:project_id>/tenant-databases/`	GET, POST	`TenantDatabaseListCreateView`	

Path	Methods	View	What it does
`/api/v1/tables/<uuid:table_id>/`	GET, DELETE	`TableDetailView`	
`/api/v1/tables/<uuid:table_id>/columns/`	POST	`ColumnCreateView`	
`/api/v1/tables/<uuid:table_id>/foreign-keys/`	GET, POST	`ForeignKeyCreateView`	
`/api/v1/tables/<uuid:table_id>/rows/`	GET, POST	`RowListCreateView`	
`/api/v1/tables/<uuid:table_id>/rows/<uuid:row_id>/`	GET, PATCH, DELETE	`RowDetailView`	
`/api/v1/tables/<uuid:table_id>/rows/export/`	GET	`RowExportView`	
`/api/v1/tenant-databases/<uuid:tenant_database_id>/`	GET, DELETE	`TenantDatabaseDetailView`	
`/api/v1/tenant-databases/<uuid:tenant_database_id>/tables/`	GET, POST	`TableListCreateView`	

environments

Path	Methods	View	What it does
`/api/v1/applications/<uuid:application_id>/environments/`	GET, POST	`EnvironmentListCreateView`	
`/api/v1/environments/<uuid:environment_id>/`	GET, PATCH, DELETE	`EnvironmentDetailView`	
`/api/v1/environments/<uuid:environment_id>/clone/`	POST	`EnvironmentCloneView`	
`/api/v1/environments/<uuid:environment_id>/credentials/`	GET, POST	`EnvironmentCredentialListCreateView`	
`/api/v1/environments/<uuid:environment_id>/credentials/<uuid:credential_id>/revoke/`	POST	`EnvironmentCredentialRevokeView`	
`/api/v1/environments/<uuid:environment_id>/credentials/<uuid:credential_id>/rotate/`	POST	`EnvironmentCredentialRotateView`	
`/api/v1/environments/<uuid:environment_id>/database/`	PATCH	`EnvironmentDatabaseBindingView`	
`/api/v1/environments/<uuid:environment_id>/disable/`	POST	`EnvironmentDisableView`	
`/api/v1/environments/<uuid:environment_id>/enable/`	POST	`EnvironmentEnableView`	

Path	Methods	View	What it does
`/api/v1/environments/<uuid:environment_id>/secrets/`	GET, POST	`EnvironmentSecretListCreateView`	
`/api/v1/environments/<uuid:environment_id>/secrets/<uuid:secret_id>/`	DELETE	`EnvironmentSecretDetailView`	
`/api/v1/environments/<uuid:environment_id>/secrets/<uuid:secret_id>/rotate/`	POST	`EnvironmentSecretRotateView`	
`/api/v1/environments/<uuid:environment_id>/storage/`	PATCH	`EnvironmentStorageBindingView`	
`/api/v1/environments/<uuid:environment_id>/variables/`	GET, POST	`EnvironmentVariableListCreateView`	
`/api/v1/environments/<uuid:environment_id>/variables/<uuid:variable_id>/`	DELETE	`EnvironmentVariableDetailView`	
`/api/v1/environments/<uuid:environment_id>/webhooks/`	GET, POST	`EnvironmentWebhookListCreateView`	
`/api/v1/environments/<uuid:environment_id>/webhooks/<uuid:webhook_id>/`	PATCH, DELETE	`EnvironmentWebhookDetailView`	

exports

Path	Methods	View	What it does
`/api/v1/export/<uuid:job_id>/`	GET	`ExportJobDetailView`	
`/api/v1/export/<uuid:job_id>/download/`	GET	`ExportJobDownloadView`	
`/api/v1/import/`	POST	`RestoreJobListCreateView`	Deliberately not organization-scoped in its URL — a restore creates a brand-new organization, so there is no existing one to nest this endpoint under (see services.stage_restore_upload).
`/api/v1/import/<uuid:job_id>/`	GET	`RestoreJobDetailView`	
`/api/v1/organizations/<uuid:organization_id>/export/`	GET, POST	`ExportJobListCreateView`	

imports

Path	Methods	View	What it does
`/api/v1/files/<uuid:file_id>/import-preview/`	GET	`ImportPreviewView`	
`/api/v1/imports/<uuid:job_id>/`	GET	`ImportJobDetailView`	

Path	Methods	View	What it does
`/api/v1/imports/<uuid:job_id>/errors/`	GET	`ImportJobErrorListView`	
`/api/v1/tables/<uuid:table_id>/imports/`	GET, POST	`ImportJobListCreateView`	

licensing

Path	Methods	View	What it does
`/api/v1/license/`	GET	`LicenceView`	
`/api/v1/license/activate/`	POST	`ActivateView`	

movie_shop

Path	Methods	View	What it does
`/api/v1/movie-ads/<uuid:ad_id>/`	PATCH, DELETE	`AdDetailView`	
`/api/v1/movie-ads/<uuid:ad_id>/picture/`	GET, POST	`AdPictureView`	
`/api/v1/movie-agent/copies/<uuid:line_id>/`	POST	`AgentCopyProgressView`	
`/api/v1/movie-agent/libraries/<uuid:library_id>/covers/`	POST	`AgentCoverView`	
`/api/v1/movie-agent/libraries/<uuid:library_id>/scan/finish/`	POST	`AgentScanFinishView`	
`/api/v1/movie-agent/libraries/<uuid:library_id>/scan/start/`	POST	`AgentScanStartView`	
`/api/v1/movie-agent/libraries/<uuid:library_id>/scan/titles/`	POST	`AgentScanTitlesView`	
`/api/v1/movie-agent/report/`	POST	`AgentReportView`	
`/api/v1/movie-customers/<uuid:customer_id>/`	PATCH	`CustomerDetailView`	

Path	Methods	View	What it does
`/api/v1/movie-customers/<uuid:customer_id>/approve/`	POST	`CustomerActionView`	
`/api/v1/movie-customers/<uuid:customer_id>/block/`	POST	`CustomerActionView`	
`/api/v1/movie-customers/<uuid:customer_id>/points/`	POST	`CustomerPointsView`	
`/api/v1/movie-customers/<uuid:customer_id>/reject/`	POST	`CustomerActionView`	
`/api/v1/movie-customers/<uuid:customer_id>/reset-code/`	POST	`CustomerActionView`	
`/api/v1/movie-customers/<uuid:customer_id>/unblock/`	POST	`CustomerActionView`	
`/api/v1/movie-customers/<uuid:customer_id>/wallet-password/`	POST	`CustomerActionView`	
`/api/v1/movie-libraries/<uuid:library_id>/`	PATCH, DELETE	`LibraryDetailView`	
`/api/v1/movie-libraries/<uuid:library_id>/scan/`	POST	`LibraryScanView`	
`/api/v1/movie-sales/<uuid:sale_id>/`	GET	`SaleDetailView`	
`/api/v1/movie-sales/<uuid:sale_id>/cancel/`	POST	`SaleActionView`	
`/api/v1/movie-sales/<uuid:sale_id>/cash/`	POST	`SaleActionView`	
`/api/v1/movie-sales/<uuid:sale_id>/charge/`	POST	`SaleChargeView`	The counter charges a portal order: the customer's drive, IF Pay or cash; their rewards apply now.

Path	Methods	View	What it does
`/api/v1/movie-sales/<uuid:sale_id>/retry/`	POST	`SaleActionView`	
`/api/v1/movie-shops/<uuid:shop_id>/`	GET, PATCH	`ShopDetailView`	
`/api/v1/movie-shops/<uuid:shop_id>/ads/`	GET, POST	`AdListCreateView`	
`/api/v1/movie-shops/<uuid:shop_id>/agent-credential/`	POST	`AgentCredentialView`	
`/api/v1/movie-shops/<uuid:shop_id>/categories/`	GET	`CategoryListView`	
`/api/v1/movie-shops/<uuid:shop_id>/counter-customers/`	GET	`CounterCustomerFindView`	For the person at the counter (movies.sell): finds an approved portal customer by phone number, to apply their rewards to a counter sale. Deliberately narrow: at least 6 digits of the number, at most 5 matches, the name, the number's last 4 digits and the poin
`/api/v1/movie-shops/<uuid:shop_id>/customers/`	GET	`CustomerListView`	
`/api/v1/movie-shops/<uuid:shop_id>/destinations/`	GET	`DestinationsView`	The drives a sold title can be copied to (what the media agent sees now), for the person at the counter.
`/api/v1/movie-shops/<uuid:shop_id>/drives/`	GET	`DrivesView`	
`/api/v1/movie-shops/<uuid:shop_id>/libraries/`	GET, POST	`LibraryListCreateView`	
`/api/v1/movie-shops/<uuid:shop_id>/portal/`	GET, PUT	`PortalSettingsView`	
`/api/v1/movie-shops/<uuid:shop_id>/prices/`	PUT	`ShopPricesView`	
`/api/v1/movie-shops/<uuid:shop_id>/sales/`	GET, POST	`SaleListCreateView`	
`/api/v1/movie-shops/<uuid:shop_id>/titles/`	GET	`TitleListView`	

Path	Methods	View	What it does
`/api/v1/movie-shops/<uuid:shop_id>/today/`	GET	`ShopTodayView`	The shop's day so far, for the catalogue's "Today" card: titles sold, credits taken and new portal customers since `since` (the browser's own midnight, so the day is the shop's, not UTC's).
`/api/v1/movie-titles/<uuid:title_id>/`	PATCH	`TitleDetailView`	
`/api/v1/movie-titles/<uuid:title_id>/cover/`	GET, POST, DELETE	`TitleCoverView`	
`/api/v1/organizations/<uuid:organization_id>/movie-shops/`	GET, POST	`ShopListCreateView`	
`/api/v1/portal/`	GET	`PortalDirectoryView`	The shop's open portals, for the short address on the shop network (https://movies.lan:8443 opens /portal/): just their names, so a phone can pick one. Nothing else about a shop is shown.
`/api/v1/portal/<shop_id>/`	GET	`PortalHomeView`	
`/api/v1/portal/<shop_id>/ads/`	GET	`PortalAdsView`	
`/api/v1/portal/<shop_id>/ads/<ad_id>/picture/`	GET	`PortalAdPictureView`	
`/api/v1/portal/<shop_id>/ads/<ad_id>/tap/`	POST	`PortalAdTapView`	
`/api/v1/portal/<shop_id>/categories/`	GET	`PortalCategoriesView`	
`/api/v1/portal/<shop_id>/forgot/`	POST	`PortalForgotView`	"I forgot my password": the owner is asked, and gives the customer a one-time code at the counter.
`/api/v1/portal/<shop_id>/login/`	POST	`PortalSignInView`	
`/api/v1/portal/<shop_id>/logout/`	POST	`PortalSignOutView`	
`/api/v1/portal/<shop_id>/me/`	GET	`PortalMeView`	
`/api/v1/portal/<shop_id>/orders/`	GET, POST	`PortalOrdersView`	
`/api/v1/portal/<shop_id>/orders/<sale_id>/cancel/`	POST	`PortalOrderCancelView`	

Path	Methods	View	What it does
`/api/v1/portal/<shop_id>/reset/`	POST	`PortalResetView`	
`/api/v1/portal/<shop_id>/signup/`	POST	`PortalSignUpView`	
`/api/v1/portal/<shop_id>/titles/`	GET	`PortalTitlesView`	
`/api/v1/portal/<shop_id>/titles/<title_id>/`	GET	`PortalTitleView`	
`/api/v1/portal/<shop_id>/titles/<title_id>/cover/`	GET	`PortalCoverView`	

oauth_provider

Path	Methods	View	What it does
`/.well-known/openid-configuration`	GET	`OpenIDConfigurationView`	
`/api/v1/oauth-clients/`	GET, POST	`OAuthClientListCreateView`	
`/api/v1/oauth-clients/<uuid:client_pk>/`	GET, PATCH	`OAuthClientDetailView`	
`/api/v1/oauth-clients/<uuid:client_pk>/rotate-secret/`	POST	`OAuthClientRotateSecretView`	
`/api/v1/oauth/consent/`	GET, POST	`ConsentView`	The consent screen's API (session only). GET says which application asks for what; POST records Allow (the screen then goes back to /oauth/authorize) or Deny (the answer is the application's redirect_uri with error=access_denied, built here only after redirect
`/oauth/authorize`	GET	`AuthorizeView`	`GET /oauth/authorize` — RFC 6749 Section 4.1.1 / OIDC Core Section 3.1.2.1. Reuses the *existing* session-cookie login/register pages verbatim for the "user must authenticate" step (redirecting to `/login?next=<this-request>`, which already round-trips throug
`/oauth/jwks.json`	GET	`JWKSView`	
`/oauth/revoke`	POST	`RevokeView`	`POST /oauth/revoke` — RFC 7009. Always 200 regardless of whether the token existed/was already revoked (Section 2.2: don't let the response distinguish an invalid token from a successfully revoked one).

Path	Methods	View	What it does
`/oauth/token`	POST	`TokenView`	`POST /oauth/token` — server-to-server (the spare-parts Next.js backend, never the browser). Accepts the client credentials in the POST body (widely supported alongside HTTP Basic; simpler for a fetch()-based caller) rather than requiring Basic auth.
`/oauth/userinfo`	GET	`UserInfoView`	`GET /oauth/userinfo` — OIDC Core Section 5.3. Bearer-token authenticated independently of DRF's configured authentication classes (this token is this app's own opaque OAuthAccessToken, not a Django session or an ApplicationCredential). `authentication_classes

org_units

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/units/`	GET, POST	`OrganizationalUnitListCreateView`	
`/api/v1/organizations/<uuid:organization_id>/units/tree/`	GET	`OrganizationalUnitTreeView`	
`/api/v1/units/<uuid:unit_id>/`	GET, PATCH, DELETE	`OrganizationalUnitDetailView`	
`/api/v1/units/<uuid:unit_id>/members/`	GET, POST	`UnitMemberListCreateView`	
`/api/v1/units/<uuid:unit_id>/members/<uuid:user_id>/`	DELETE	`UnitMemberDetailView`	

organizations

Path	Methods	View	What it does
`/api/v1/organizations/`	GET, POST	`OrganizationListCreateView`	
`/api/v1/organizations/<uuid:organization_id>/`	GET	`OrganizationDetailView`	
`/api/v1/organizations/<uuid:organization_id>/members/`	GET, POST	`MembershipListCreateView`	
`/api/v1/organizations/<uuid:organization_id>/members/<uuid:membership_id>/reset-password/`	POST	`MembershipPasswordResetView`	An administrator gives a staff member a temporary password (shown once) when they forgot theirs: users.manage here, and in every other organization the person belongs to (accounts.services.can_reset_password). The person is signed out everywhere and must choos

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/members/<uuid:membership_id>/role/`	POST	`MembershipRoleAssignView`	
`/api/v1/organizations/<uuid:organization_id>/members/<uuid:membership_id>/roles/<slug:role_slug>/`	DELETE	`MembershipRoleRemoveView`	
`/api/v1/organizations/<uuid:organization_id>/roles/`	GET	`OrganizationRolesView`	The roles this organization can give its members, and who holds which. Only for people who may manage roles (permissions.manage): who has which role is not shown to every member.
`/api/v1/organizations/<uuid:organization_id>/teams/`	GET, POST	`TeamListCreateView`	
`/api/v1/teams/<uuid:team_id>/members/`	POST	`TeamMemberListCreateView`	
`/api/v1/teams/<uuid:team_id>/members/<uuid:user_id>/`	DELETE	`TeamMemberDetailView`	

service_apps

Path	Methods	View	What it does
`/api/v1/internal/service-apps/audit-events/`	POST	`InternalAuditView`	
`/api/v1/internal/service-apps/config/`	GET, PATCH	`InternalConfigView`	
`/api/v1/internal/service-apps/files/`	POST	`InternalFilesView`	
`/api/v1/internal/service-apps/files/<uuid:file_id>/`	GET, DELETE	`InternalFileDetailView`	
`/api/v1/internal/service-apps/heartbeat/`	POST	`InternalHeartbeatView`	
`/api/v1/internal/service-apps/pay/payment-requests/`	POST	`InternalPayRequestsView`	Ask this shop's IntraForge Pay to take a payment (pay_bridge.py): the caller names the person, the amount, what it is for and its own record; IntraForge picks the Pay and checks the person.

Path	Methods	View	What it does
`/api/v1/internal/service-apps/pay/payment-requests/<uuid:request_id>/`	GET	`InternalPayRequestDetailView`	
`/api/v1/internal/service-apps/self/`	GET	`InternalSelfView`	
`/api/v1/internal/service-apps/sessions/introspect/`	POST	`InternalIntrospectView`	
`/api/v1/internal/service-apps/sessions/revoke/`	POST	`InternalRevokeView`	
`/api/v1/internal/service-apps/users/`	GET	`InternalUsersView`	
`/api/v1/pay/payment-requests/`	POST	`AppPayRequestsView`	
`/api/v1/pay/payment-requests/<uuid:request_id>/`	GET	`AppPayRequestDetailView`	
`/api/v1/service-apps/`	GET, POST	`ServiceAppListView`	
`/api/v1/service-apps/<uuid:app_id>/`	GET, PATCH, DELETE	`ServiceAppDetailView`	
`/api/v1/service-apps/<uuid:app_id>/launch/`	POST	`ServiceAppLaunchView`	"Open": mints an application session bound to this IntraForge login and sets it as an HttpOnly, SameSite=Strict cookie on the application's path. Nothing goes in the URL.
`/api/v1/service-apps/<uuid:app_id>/reinstate/`	POST	`ServiceAppReinstateView`	
`/api/v1/service-apps/<uuid:app_id>/removal-preview/`	GET	`ServiceAppRemovalPreviewView`	What removing would affect (counts only), so the confirmation can say exactly what goes.
`/api/v1/service-apps/<uuid:app_id>/sign-in/`	GET	`ServiceAppSignInInfoView`	What the sign-in page shows when an application sent someone there ("Sign in to Spare Parts -- <shop>"). Public, but only what the application itself already shows before sign-in: its product name and shop name. Unknown, disabled or removed instances are a pla

sharing

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/external-sharing/`	PATCH	`ExternalSharingSettingView`	
`/api/v1/organizations/<uuid:organization_id>/shares/`	GET, POST	`ShareGrantListCreateView`	
`/api/v1/shares/<uuid:share_id>/`	GET, DELETE	`ShareGrantDetailView`	

sites

Path	Methods	View	What it does
`/_sites-certificate/`	-	`certificate_allowed`	The proxy asks before it issues a certificate for a name (on-demand TLS): an online site's own name, or one of IntraForge's own names (OWN_TLS_HOSTNAMES), gets one. The proxy already serves IntraForge's names with its internal certificate; it also asks here wh
`/_sites/`	-	`sites_index`	The bare sites port and anything that is not a site address. No list of sites is given out.
`/_sites/^(?P<path>.*)$`	-	`sites_index`	The bare sites port and anything that is not a site address. No list of sites is given out.
`/_sites/^(?P<slug>[a-z0-9-]{1,50})$`	-	`site_root`	`/<slug>` without the slash: relative links in index.html need the slash.
`/_sites/^(?P<slug>[a-z0-9-]{1,50})/(?P<path>.*)$`	-	`serve_site`	
`/_sites/^_host/(?P<path>.*)$`	-	`serve_by_host`	A site under its own name (https://<name>:<sites port>/...): the proxy says which name was asked for.
`/_sites/^_internet/(?P<path>.*)$`	-	`serve_internet`	A site published to the internet, reached through the tunnel and the proxy's internet listener only while the gate is on (ADR-0020 decision 4).
`/api/v1/internet-gateway/`	GET	`InternetGatewayView`	The gate's state. Everyone signed in may see whether it is on; only the Super Administrator sees the tunnel and what is published.
`/api/v1/internet-gateway/auto-publish/`	POST	`InternetAutoPublishView`	Auto-publish under a domain ({"domain": "harmongt.uk", "password": ...}), or off ({"domain": ""}).
`/api/v1/internet-gateway/disable/`	POST	`InternetGatewayDisableView`	
`/api/v1/internet-gateway/enable/`	POST	`InternetGatewayEnableView`	

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/sites/`	GET, POST	`SiteListCreateView`	
`/api/v1/sites/<uuid:site_id>/`	GET, PATCH, DELETE	`SiteDetailView`	
`/api/v1/sites/<uuid:site_id>/internet/`	POST, DELETE	`SiteInternetView`	Publish a site under a public name, or revoke it. The Super Administrator acts platform-wide, so the site is looked up directly -- but only for them; anyone else is refused before any lookup.
`/api/v1/sites/<uuid:site_id>/logs/`	GET	`SiteLogsView`	A server app's own output, its last 200 lines (sites.manage: an app's log can hold what its settings hold, so reading the site is not enough).
`/api/v1/sites/<uuid:site_id>/versions/`	GET, POST	`SiteVersionListCreateView`	
`/api/v1/sites/<uuid:site_id>/versions/<uuid:version_id>/`	DELETE	`SiteVersionDetailView`	
`/api/v1/sites/<uuid:site_id>/versions/<uuid:version_id>/activate/`	POST	`SiteVersionActivateView`	

storage

Path	Methods	View	What it does
`/api/v1/buckets/<uuid:bucket_id>/`	GET, PATCH	`BucketDetailView`	One bucket's name and project, for a page opened straight from its address (a bookmark, a refresh): the same checks as listing its files.
`/api/v1/buckets/<uuid:bucket_id>/files/`	GET, POST	`FileListCreateView`	
`/api/v1/buckets/<uuid:bucket_id>/folders/`	GET, POST	`FolderListCreateView`	
`/api/v1/files/<uuid:file_id>/`	GET, PATCH, DELETE	`FileDetailView`	
`/api/v1/files/<uuid:file_id>/download/`	GET	`FileDownloadView`	Streams the file through the backend rather than issuing a presigned URL directly to MinIO — OBJECT_STORAGE_ENDPOINT is an internal Docker DNS name, unreachable from a browser outside the Docker network in this deployment topology. See storage/backends.py:Obje
`/api/v1/files/<uuid:file_id>/restore/`	POST	`FileRestoreView`	

Path	Methods	View	What it does
`/api/v1/files/<uuid:file_id>/share-links/`	GET, POST	`FileShareLinkListCreateView`	A file's share links (storage.share on its bucket). POST {days, max_downloads?} makes one; its address is in that answer only.
`/api/v1/files/<uuid:file_id>/versions/`	GET, POST	`FileVersionUploadView`	
`/api/v1/files/<uuid:file_id>/versions/<uuid:version_id>/download/`	GET	`FileVersionDownloadView`	Downloads one kept earlier content of a file (storage.read), streamed like the current one.
`/api/v1/files/<uuid:file_id>/versions/<uuid:version_id>/restore/`	POST	`FileVersionRestoreView`	Makes an earlier content the current one (storage.write); the replaced content is kept.
`/api/v1/organizations/<uuid:organization_id>/buckets/`	GET	`OrganizationBucketListView`	Every bucket in an organization the caller may read (storage.read, role-wide or a grant on that bucket, the same check as each project's list), with what it holds: files in use, their bytes, the bytes of kept earlier versions, and the Trash. For the organizati
`/api/v1/projects/<uuid:project_id>/buckets/`	GET, POST	`BucketListCreateView`	
`/api/v1/share-links/<uuid:link_id>/revoke/`	POST	`FileShareLinkRevokeView`	
`/api/v1/share/<str:token>/`	GET	`PublicShareView`	What a share link opens, for anyone who has it: the file's name and size (GET), or the file itself (download/). Nothing else about the organization is shown; a link that does not work is a plain 404.
`/api/v1/share/<str:token>/download/`	GET	`PublicShareView`	What a share link opens, for anyone who has it: the file's name and size (GET), or the file itself (download/). Nothing else about the organization is shown; a link that does not work is a plain 404.

system

Path	Methods	View	What it does
`/api/v1/system/backups/`	GET	`BackupListView`	
`/api/v1/system/backups/run/`	POST	`BackupRunView`	
`/api/v1/system/health/`	GET	`HealthView`	Health history and alerts for this installation (Super Administrator only).

Path	Methods	View	What it does
`/api/v1/system/version/`	GET	`VersionView`	The installed IntraForge version (the repo-root VERSION file, as the portable export records it), for the web app's footer and welcome tour. Signed-in callers only: an anonymous caller is never told the version.
`/healthz`	GET	`HealthzView`	Liveness: process is up and can respond. No dependency checks — used by the container runtime/orchestrator to know whether to restart the process, not whether it's ready to serve real traffic.
`/metrics`	GET	`MetricsView`	Prometheus exposition format (Phase 11; docs/architecture/ROADMAP.md). Exposes dependency-up gauges computed at scrape time from the same checks /readyz runs. Deliberately does not expose request-count/ latency histograms — those need a registry shared across
`/readyz`	GET	`ReadyzView`	Readiness: checks the dependencies a request would actually need (control-plane DB, tenant DB connection, Valkey) before the process is considered ready to receive traffic (docs/architecture/ARCHITECTURE.md Section 9).

workspaces

Path	Methods	View	What it does
`/api/v1/organizations/<uuid:organization_id>/workspaces/`	GET, POST	`WorkspaceListCreateView`	
`/api/v1/projects/<uuid:project_id>/`	GET	`ProjectDetailView`	
`/api/v1/workspaces/<uuid:workspace_id>/`	GET	`WorkspaceDetailView`	
`/api/v1/workspaces/<uuid:workspace_id>/projects/`	GET, POST	`ProjectListCreateView`	

Reference B — the data model

Every model of IntraForge's own apps, generated from the models: table, fields (type; null; relation) and the model's description. Tenant business data (tables people create, installed apps' records) lives in per-organization PostgreSQL schemas, not here (ADR-0005).

accounts

User (`accounts_user`)

Field	Type	Notes
`password`	CharField	
`last_login`	DateTimeField	null
`id`	UUIDField	primary key
`email`	EmailField	unique
`first_name`	CharField	
`last_name`	CharField	
`is_active`	BooleanField	
`date_joined`	DateTimeField	
`mfa_enabled`	BooleanField	
`must_change_password`	BooleanField	
`mfa_secret_encrypted`	BinaryField	null
`mfa_confirmed_at`	DateTimeField	null
`passkey_only`	BooleanField	

Passkey (`accounts_passkey`)

A WebAuthn credential (ADR-0019). Belongs to one user and to one relying-party ID -- the host name the browser used when it was made (`localhost`, the shop-network name): it signs in only at that name.

Field	Type	Notes
`id`	UUIDField	primary key
`user`	ForeignKey	→ User
`name`	CharField	
`rp_id`	CharField	
`credential_id`	BinaryField	unique
`public_key`	BinaryField	
`sign_count`	PositiveBigIntegerField	

Field	Type	Notes
`transports`	JSONField	
`backed_up`	BooleanField	
`created_at`	DateTimeField	
`last_used_at`	DateTimeField	null

MFARecoveryCode (`accounts_mfarecoverycode`)

One of a person's one-time recovery codes for two-step sign-in, for when their authenticator is lost. Only a hash is kept; the codes are shown once, when made.

Field	Type	Notes
`id`	UUIDField	primary key
`user`	ForeignKey	→ User
`code_hash`	CharField	
`used_at`	DateTimeField	null
`created_at`	DateTimeField	

analytics

Dashboard (`analytics_dashboard`)

A saved arrangement of analytics widgets (Section 25 of the master prompt). Widgets are stored as declarative JSON — {table_id, operation, params, chart_type, title, position} — never a saved raw query, so `views.py` 's render endpoint can re-validate every widget's permissions and re-run its operation against live data every time it's viewed, rather than trusting a cached result a later-revoked grant should have invalidated.

Field	Type	Notes
`id`	UUIDField	primary key
`tenant_database`	ForeignKey	→ TenantDatabase
`name`	CharField	
`widgets`	JSONField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

app_platform

AppTemplate (`app_platform_apptemplate`)

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`organization`	ForeignKey	→ Organization
`description`	TextField	
`draft`	JSONField	
`archived`	BooleanField	
`created_by`	ForeignKey	→ User

AppTemplateVersion (`app_platform_apptemplateversion`)

Field	Type	Notes
`id`	UUIDField	primary key
`template`	ForeignKey	→ AppTemplate
`number`	PositiveIntegerField	
`definition`	JSONField	
`checksum_sha256`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

AppInstance (`app_platform_appinstance`)

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`organization`	ForeignKey	→ Organization
`project`	ForeignKey	→ Project
`source_version`	ForeignKey	→ AppTemplateVersion
`archived`	BooleanField	
`created_by`	ForeignKey	→ User

ModelDefinition (`app_platform_modeldefinition`)

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	

Field	Type	Notes
`created_at`	DateTimeField	
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`instance`	ForeignKey	→ AppInstance

FieldDefinition (`app_platform_fielddefinition`)

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`model`	ForeignKey	→ ModelDefinition
`data_type`	CharField	choices: text, integer, decimal, boolean, date, datetime, org_unit_reference
`required`	BooleanField	
`default_value`	JSONField	null
`unique`	BooleanField	
`indexed`	BooleanField	

ConstraintDefinition (`app_platform_constraintdefinition`)

A model-level composite UNIQUE constraint -- Post-Phase-3 Integration Enablement's general composite-uniqueness capability (docs/SPARE_PARTS_INTEGRATION_READINESS.md's "Inventory (Part x Warehouse)" row). Membership is `fields` union `relationships`, >=2 combined, all belonging to this same `model` (enforced by definitions.py/instances.py, never by a DB constraint spanning two tables). A relationship is only eligible when its own FK column lives on this model's table -- i.e. `relationship.source_model == model` and `relationship.kind == "many_to_one"`; a `many_to_many` relationship has no column on either side (it's a join table) and can never participate. Physical enforcement is a real Postgres UNIQUE constraint added via databases.services.add_field_set_unique_constraint, keyed by this row's own stable `id` -- never by the member set or by `label` -- so a display-label rename or a member's position/order changing never touches DDL or requires reprovisioning.

Field	Type	Notes
`id`	UUIDField	primary key

Field	Type	Notes
`label`	CharField	
`created_at`	DateTimeField	
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`model`	ForeignKey	→ ModelDefinition
`fields`	ManyToManyField	→ FieldDefinition
`relationships`	ManyToManyField	→ RelationshipDefinition

RelationshipDefinition (`app_platform_relationshipdefinition`)

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`instance`	ForeignKey	→ AppInstance
`source_model`	ForeignKey	→ ModelDefinition
`target_model`	ForeignKey	→ ModelDefinition
`kind`	CharField	
`deletion_policy`	CharField	

WorkflowDefinition (`app_platform_workflowdefinition`)

Phase 5: a declarative state machine over one of `model`'s own existing text fields -- there is no enum type in this platform, so `state_field` (validated text-typed, enforced in definitions.py/ instances.py, never by a DB constraint spanning two tables the way FieldDefinition.data_type is) holds whatever value the workflow's current state says it should. Like every Definition, `position` is excluded from runtime_plan.py's fingerprint -- but unlike every other concrete Definition subclass, the *whole* workflow subsystem is excluded from it: no migration or provisioning code ever reads these tables. It's pure control-plane declaration, governing legal writes to an already-physical field, never new schema of its own.

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	

Field	Type	Notes
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`model`	ForeignKey	→ ModelDefinition
`state_field`	ForeignKey	→ FieldDefinition

WorkflowState (`app_platform_workflowstate`)

A legal value of its workflow's `state_field`. Deliberately no separate `value` column -- `key` is already a validated, storage-safe identifier (KeyField), and reusing it as the literal text written to the record avoids a redundant, driftable second name for the same thing.

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`key`	CharField	
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`workflow`	ForeignKey	→ WorkflowDefinition

WorkflowTransition (`app_platform_workflowtransition`)

One legal move from `from_state` (null = valid from any current state) to `to_state`. `conditions` is a JSON list of `{field_id, operator, value}` (operator in eq/ne/is_null/is_not_null), ANDed -- deliberately as narrow as databases/rows.py's own exact-match-only filtering, not a general expression language. `field_actions` is a JSON list of `{field_id, value}` literal writes applied in the exact same UPDATE as the state change (databases.rows.update_row_if) -- literal-only (no calculated values) is what makes a replayed apply naturally idempotent: setting a column to a fixed value twice is a no-op the second time, unlike e.g. incrementing a counter. `requires_approval` gates this specific transition behind the `app_instance.workflow.approve` capability, in addition to the `database.write` every record write already needs -- see docs/implementation/APP_PLATFORM_PHASE5.md for why this is a capability gate and not a stateful pending/approve workflow. `notify_url`, if set, is validated against databases.connectors.assert_host_is_safe at definition time (and re-checked immediately before every delivery attempt, since DNS can rebind between the two).

Field	Type	Notes
`id`	UUIDField	primary key
`label`	CharField	
`created_at`	DateTimeField	
`key`	CharField	

Field	Type	Notes
`source_definition_id`	UUIDField	null
`position`	PositiveIntegerField	
`workflow`	ForeignKey	→ WorkflowDefinition
`from_state`	ForeignKey	null; → WorkflowState
`to_state`	ForeignKey	→ WorkflowState
`conditions`	JSONField	
`field_actions`	JSONField	
`requires_approval`	BooleanField	
`notify_url`	TextField	

RuntimeProvision (`app_platform_runtimeprovision`)

Durable reservation and immutable publication of one runtime per instance.

Field	Type	Notes
`instance`	OneToOneField	primary key; → AppInstance
`plan`	JSONField	
`fingerprint`	CharField	
`database`	OneToOneField	null; unique; → TenantDatabase
`bindings`	JSONField	
`last_error`	CharField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	
`completed_at`	DateTimeField	null

RecordAttachment (`app_platform_recordattachment`)

Links an existing `storage.FileObject` to one record. The record itself lives in a raw tenant table this app never models in Django, so `record_id` is a plain UUID, not a foreign key -- its only integrity guarantee is `records.delete_record` also removing rows here, not a database constraint (see that function's own docstring for why this can't be a single atomic operation across the control/tenant databases).

Field	Type	Notes
`id`	UUIDField	primary key
`model`	ForeignKey	→ ModelDefinition
`record_id`	UUIDField	
`file`	ForeignKey	→ FileObject
`created_by`	ForeignKey	→ User

Field	Type	Notes
`created_at`	DateTimeField	

WorkflowTransitionAttempt (`app_platform_workflowtransitionattempt`)

A durable receipt for one "apply this transition to this record" request -- operational state, not declarative metadata (deliberately not a Definition subclass). Mirrors RuntimeProvision/exports.RestoreJob's own reserve-then-execute shape: `completed_at` is the single durable "already fully applied" marker a replayed request short-circuits on (workflows.py's apply_transition checks it before redoing any work), and `idempotency_key` + `request_fingerprint` give the exact same actor-opts-in idempotency RestoreJob's own docstring describes -- a repeat with the same key AND the same fingerprint returns the original outcome; the same key with a different fingerprint is a genuine conflicting request, rejected rather than silently reinterpreted. `record_id` is a bare UUID, not a FK, the same cross-database precedent RecordAttachment.record_id already establishes -- the record itself lives in a raw tenant table this app never models in Django.

Field	Type	Notes
`id`	UUIDField	primary key
`transition`	ForeignKey	→ WorkflowTransition
`record_id`	UUIDField	
`idempotency_key`	UUIDField	null
`request_fingerprint`	CharField	
`state_before`	CharField	
`state_after`	CharField	
`completed_at`	DateTimeField	null
`last_error`	CharField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	

WorkflowNotificationDelivery (`app_platform_workflownotificationdelivery`)

An at-least-once outbox row for one transition attempt's notification -- one row per attempt that had a `notify_url` at the moment it completed. `url` is a denormalized copy taken at that moment, not a live read of the transition's current `notify_url`, so editing a transition's notification target later never rewrites history. `delivered_at` is the exact same kind of durable completion marker WorkflowTransitionAttempt.completed_at is -- the Celery delivery task (app_platform/tasks.py) checks it before ever dialing out. Remote acceptance before that marker commits can cause a repeat; receivers must deduplicate the stable event_id.

Field	Type	Notes
`id`	UUIDField	primary key
`attempt`	ForeignKey	→ WorkflowTransitionAttempt

Field	Type	Notes
`url`	TextField	
`delivered_at`	DateTimeField	null
`attempt_count`	PositiveIntegerField	
`last_error`	CharField	
`created_at`	DateTimeField	
`next_attempt_at`	DateTimeField	
`exhausted_at`	DateTimeField	null

WebhookDelivery (`app_platform_webhookdelivery`)

Phase 10: a durable at-least-once outbox row for one `environments.EnvironmentWebhook` delivery of an App Platform event (`app_platform/webhooks.py`) -- the "reliable webhooks" this phase's own exit bar names, and the concrete mechanism `docs/ EXTERNAL_APP_API_CONTRACT.md` previously flagged outright as **NOT YET IMPLEMENTED** ("Webhooks or any push/event-notification mechanism out of App Platform toward an external application").

Field	Type	Notes
`id`	UUIDField	primary key
`webhook`	ForeignKey	→ EnvironmentWebhook
`instance`	ForeignKey	→ AppInstance
`event_type`	CharField	
`payload`	JSONField	
`delivered_at`	DateTimeField	null
`attempt_count`	PositiveIntegerField	
`last_error`	CharField	
`created_at`	DateTimeField	
`next_attempt_at`	DateTimeField	
`exhausted_at`	DateTimeField	null

UnitScopedGrant (`app_platform_unitscopedgrant`)

Phase 7 Step 3 ("unit scope"): grants one user read/write access to a model's records, restricted to records whose org_unit_reference field equals `unit` . Real FKs to both `model` and `unit` -- unlike org_unit_reference itself, both rows here live in the control-plane database (this app and org_units), so there's no cross-database problem to work around.

Field	Type	Notes
`id`	UUIDField	primary key
`model`	ForeignKey	→ ModelDefinition

Field	Type	Notes
`unit`	ForeignKey	→ OrganizationalUnit
`user`	ForeignKey	→ User
`level`	CharField	choices: read, write
`granted_by`	ForeignKey	null; → User
`expires_at`	DateTimeField	null
`created_at`	DateTimeField	

AppInstanceUpgrade (`app_platform_appinstanceupgrade`)

Phase 9: a durable receipt for one `upgrades.apply_upgrade` call -- operational history, not declarative metadata (deliberately not a `Definition` subclass, the same distinction `WorkflowTransitionAttempt` already draws above). `summary` is the full three-way diff this call computed (see `upgrades.py`): every item added or label-synced, and every item that needed a human because it conflicted with a local customization, changed something a live migration can't safely apply, or was removed upstream -- recorded even when it wasn't applied, never silently dropped. There is nothing here to *resume*: `apply_upgrade` runs entirely inside one control-plane transaction (plus one nested tenant transaction for any real DDL), so a crash mid-call leaves nothing partially applied -- Postgres's own transactional DDL is the recovery mechanism, matching every other schema_evolution.py caller (RuntimeProvision/WorkflowTransitionAttempt need their own resumable-job machinery only because they span an async Celery task; this doesn't).

Field	Type	Notes
`id`	UUIDField	primary key
`instance`	ForeignKey	→ AppInstance
`from_version`	ForeignKey	→ AppTemplateVersion
`to_version`	ForeignKey	→ AppTemplateVersion
`summary`	JSONField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	

QuantityLedgerBinding (`app_platform_quantityledgerbinding`)

ADR-0016's closing half: an opt-in, administrator-validated binding between an append-only movement model's field and the balance model's field its inserts must atomically maintain, over a shared set of relationship "dimensions" (e.g. Part, Warehouse) -- `operation_journal.py` covers durable audit/webhook evidence for a mutation that already committed; this covers making the mutation itself atomic (and concurrency-safe, and rejecting an insufficient balance) in the first place, enforced by a real Postgres trigger installed by `quantity_ledger.bind()`, not application code -- so it holds even for a direct SQL insert that never goes through `records.py`.

Field	Type	Notes
`id`	UUIDField	primary key

Field	Type	Notes
`instance`	ForeignKey	→ AppInstance
`balance_model`	ForeignKey	→ ModelDefinition
`balance_field`	ForeignKey	→ FieldDefinition
`unique_constraint`	ForeignKey	→ ConstraintDefinition
`movement_model`	ForeignKey	→ ModelDefinition
`movement_field`	ForeignKey	→ FieldDefinition
`allow_negative`	BooleanField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	

QuantityLedgerDimension (`app_platform_quantityledgerdimension`)

One (balance relationship, movement relationship) pair identifying the same real-world dimension on both sides of a QuantityLedgerBinding. Both must be many_to_one relationships whose own `target_model` matches, and whose `source_model` is the binding's `balance_model` / `movement_model` respectively -- validated in `quantity_ledger.bind()`, not by a DB constraint spanning definitions this way, the same discipline `ConstraintDefinition`'s own docstring already documents for its cross-table membership rules.

Field	Type	Notes
`id`	UUIDField	primary key
`binding`	ForeignKey	→ QuantityLedgerBinding
`balance_relationship`	ForeignKey	→ RelationshipDefinition
`movement_relationship`	ForeignKey	→ RelationshipDefinition

applications

Application (`applications_application`)

A registered piece of software, owned by an Organization, with an owning User (docs/architecture/DATA_MODEL.md Section 3.7).

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`name`	CharField	
`description`	TextField	
`owner`	ForeignKey	→ User
`created_at`	DateTimeField	

ServiceAccount (`applications_serviceaccount`)

The identity an Application authenticates as. Deliberately backed by a real `accounts.User` row (`identity_user`, unusable password — it can never log into the web UI) rather than a parallel principal type: every existing permission check, RoleAssignment, ResourceGrant, and tenant-membership lookup already works in terms of `User`, and duplicating that machinery for a second principal type would be a second, easy-to-miss place for the isolation invariants in docs/security/THREAT_MODEL.md Section 4 to be re-implemented (and potentially get wrong). A Membership row (created alongside this one, see applications/services.py) makes the identity_user resolvable by every existing get_member_* helper; Membership alone grants no permissions, so this doesn't widen access — only RoleAssignment/ ResourceGrant do that, exactly as for a human user.

Field	Type	Notes
`id`	UUIDField	primary key
`application`	OneToOneField	unique; → Application
`identity_user`	OneToOneField	unique; → User
`created_at`	DateTimeField	

ApplicationCredential (`applications_applicationcredential`)

Never stores the plaintext secret — only its SHA-256 hash (fast hash, not a slow password hasher: the secret is a high-entropy random token generated by the server, not a user-chosen password, so brute-force/rainbow-table resistance from a slow hash isn't the relevant property here — Section 13 of the master prompt: never store plaintext long-term secrets when a secure hash is sufficient). The plaintext is returned exactly once, at creation/rotation time, by applications/services.py — never persisted, never logged.

Field	Type	Notes
`id`	UUIDField	primary key
`service_account`	ForeignKey	→ ServiceAccount
`environment`	ForeignKey	null; → Environment
`secret_hash`	CharField	unique
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	
`last_used_at`	DateTimeField	null
`expires_at`	DateTimeField	null
`revoked_at`	DateTimeField	null

ApiRequestLog (`applications_apirequestlog`)

One request an application made with its bearer credential, for the application's Requests page: what it called and how it went. Never the query string, a body or a header (they can carry data or the secret itself). Kept API_REQUEST_LOG_DAYS days (applications.tasks.prune_api_request_log).

Field	Type	Notes
`id`	BigAutoField	primary key
`application`	ForeignKey	→ Application
`credential`	ForeignKey	null; → ApplicationCredential
`method`	CharField	
`path`	CharField	
`status`	PositiveSmallIntegerField	
`duration_ms`	PositiveIntegerField	
`at`	DateTimeField	

audit

AuditEvent (`audit_auditevent`)

Field	Type	Notes
`id`	UUIDField	primary key
`timestamp`	DateTimeField	
`actor`	ForeignKey	null; → User
`organization`	ForeignKey	null; → Organization
`action`	CharField	
`resource_type`	CharField	
`resource_id`	CharField	
`request_id`	CharField	
`result`	CharField	choices: success, denied, error
`context`	JSONField	

catalogue_storage

OEMPart (`catalogue_storage_oempart`)

Answers "what part is this" — never a selling price, stock level, or anything else that answers "what will we sell it for" (that's the spare-parts application's own CustomerCatalogueItem, deliberately not duplicated here — see the module docstring).

Field	Type	Notes
`id`	UUIDField	primary key
`manufacturer`	CharField	
`part_number`	CharField	

Field	Type	Notes
`normalized_part_number`	CharField	unique
`description`	CharField	
`applicability`	CharField	
`fitment_notes`	CharField	
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

Diagram (`catalogue_storage_diagram`)

Field	Type	Notes
`id`	UUIDField	primary key
`provider`	CharField	
`provider_diagram_id`	CharField	
`diagram_code`	CharField	
`file`	ForeignKey	null; → FileObject
`source_image_url`	URLField	
`figure_data`	JSONField	
`license_status`	CharField	choices: EXTERNAL_REFERENCE_ONLY, LOCALLY_STORED
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

DiagramPart (`catalogue_storage_diagrampart`)

One diagram can show many parts; one part can appear on more than one diagram (a different production run's exploded view, for instance) -- this is a genuine many-to-many, not a FK on either side. `x` / `y` / `width` / `height` are the provider's own real hotspot rectangle when it supplies one; left null otherwise -- never fabricated (see docs/CATALOGUE_STORAGE.md "Image highlighting").

Field	Type	Notes
`id`	UUIDField	primary key
`diagram`	ForeignKey	→ Diagram
`part`	ForeignKey	→ OEMPart
`callout`	CharField	
`x`	FloatField	null
`y`	FloatField	null
`width`	FloatField	null
`height`	FloatField	null
`created_at`	DateTimeField	

databases

TenantDatabase (`databases_tenantdatabase`)

A logical database the platform manages for a Project. Maps 1:1 to one physical PostgreSQL schema in the tenant cluster (docs/architecture/adr/0005-tenant-database-isolation-strategy.md). `schema_name` is server-generated from this row's own UUID — never derived from user input (Section 5/8 of the master prompt).

Field	Type	Notes
`id`	UUIDField	primary key
`project`	ForeignKey	→ Project
`environment`	OneToOneField	null; unique; → Environment
`name`	CharField	
`schema_name`	CharField	unique
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

DBTable (`databases_dbtable`)

Field	Type	Notes
`id`	UUIDField	primary key
`tenant_database`	ForeignKey	→ TenantDatabase
`name`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

DBColumn (`databases_dbcolumn`)

Field	Type	Notes
`id`	UUIDField	primary key
`table`	ForeignKey	→ DBTable
`name`	CharField	
`data_type`	CharField	choices: text, varchar, integer, bigint, decimal, boolean, date, datetime, uuid, json
`max_length`	PositiveIntegerField	null
`precision`	PositiveIntegerField	null
`scale`	PositiveIntegerField	null
`is_primary_key`	BooleanField	
`is_nullable`	BooleanField	
`is_unique`	BooleanField	

Field	Type	Notes
`default_value`	JSONField	null
`created_at`	DateTimeField	

DBForeignKey (`databases_dbforeignkey`)

Field	Type	Notes
`id`	UUIDField	primary key
`column`	OneToOneField	unique; → DBColumn
`references_table`	ForeignKey	→ DBTable
`references_column`	ForeignKey	→ DBColumn
`on_delete`	CharField	choices: cascade, restrict, set_null
`created_at`	DateTimeField	

ConnectedDatabase (`databases_connecteddatabase`)

An externally-hosted database reached in **connected mode**: the platform proxies authenticated read queries to it; nothing is copied in. Deliberately a distinct model from `TenantDatabase` — a single connection is never simultaneously "connected" and "imported" (docs/architecture/adr/0009-external-database-connector-modes.md). `encrypted_password` is a Fernet token (`databases/crypto.py`); the plaintext password is never stored and never returned by any API response.

Field	Type	Notes
`id`	UUIDField	primary key
`project`	ForeignKey	→ Project
`name`	CharField	
`engine`	CharField	choices: postgresql
`host`	CharField	
`port`	PositiveIntegerField	
`database_name`	CharField	
`username`	CharField	
`encrypted_password`	BinaryField	
`sslmode`	CharField	choices: disable, prefer, require, verify-full
`status`	CharField	choices: untested, connected, unreachable
`last_tested_at`	DateTimeField	null
`last_test_error`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

DBIndex (`databases_dbindex`)

Created automatically alongside unique columns and primary keys — not (yet) a user-facing "create an arbitrary index" feature; still a real catalog entry mirroring what actually exists in the tenant schema, per docs/architecture/DATA_MODEL.md Section 3.5.

Field	Type	Notes
`id`	UUIDField	primary key
`table`	ForeignKey	→ DBTable
`name`	CharField	unique
`is_unique`	BooleanField	
`created_at`	DateTimeField	
`columns`	ManyToManyField	→ DBColumn

environments

Environment (`environments_environment`)

Field	Type	Notes
`id`	UUIDField	primary key
`application`	ForeignKey	→ Application
`name`	CharField	
`slug`	SlugField	
`environment_type`	CharField	choices: development, staging, production, custom
`is_production_tier`	BooleanField	
`status`	CharField	choices: active, disabled
`config`	JSONField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	
`last_activity_at`	DateTimeField	null

EnvironmentVariable (`environments_environmentvariable`)

Plain, non-secret configuration (e.g. LOG_LEVEL=debug). Secret values never belong here — see EnvironmentSecret.

Field	Type	Notes
`id`	UUIDField	primary key
`environment`	ForeignKey	→ Environment

Field	Type	Notes
`key`	CharField	
`value`	TextField	
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

EnvironmentSecret (`environments_environmentsecret`)

Never stores a plaintext value -- only a Fernet ciphertext (environments/crypto.py, same CREDENTIAL_ENCRYPTION_KEY-derived key as accounts/databases' equivalent modules). The plaintext is known only at creation/rotation time, in the API response body for that one request, and is never logged, never included in an audit event context, and never re-displayed by any read endpoint afterward.

Field	Type	Notes
`id`	UUIDField	primary key
`environment`	ForeignKey	→ Environment
`key`	CharField	
`value_ciphertext`	BinaryField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	
`rotated_at`	DateTimeField	null

EnvironmentWebhook (`environments_environmentwebhook`)

The signing secret is encrypted the same way EnvironmentSecret's values are -- never returned by a read endpoint after creation.

Field	Type	Notes
`id`	UUIDField	primary key
`environment`	ForeignKey	→ Environment
`url`	URLField	
`event_types`	JSONField	
`enabled`	BooleanField	
`signing_secret_ciphertext`	BinaryField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

exports

ExportJob (`exports_exportjob`)

One .icp portable-export package build for one Organization (the only export_type implemented so far — Section 14 of the master prompt lists several narrower scopes as future work). The finished package is written to object storage, not the control-plane database — this row is metadata/status only (ADR-0001).

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`created_by`	ForeignKey	→ User
`status`	CharField	choices: pending, running, completed, failed
`encrypted`	BooleanField	
`object_key`	CharField	
`size_bytes`	BigIntegerField	null
`checksum_sha256`	CharField	
`error_message`	CharField	
`created_at`	DateTimeField	
`completed_at`	DateTimeField	null

RestoreJob (`exports_restorejob`)

One .icp import/restore attempt. `organization` is null until the restore actually creates the new Organization (Section 17 of the master prompt: nothing should be visible/reachable until the whole restore either fully commits or fully rolls back) — see exports/restorer.py.

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	null; → Organization
`created_by`	ForeignKey	→ User
`status`	CharField	choices: pending, validating, restoring, completed, failed
`source_object_key`	CharField	
`recovery_version`	PositiveSmallIntegerField	
`source_sha256`	CharField	
`idempotency_key`	UUIDField	null
`request_fingerprint`	CharField	
`report`	JSONField	
`error_message`	CharField	
`created_at`	DateTimeField	
`completed_at`	DateTimeField	null

hosted_sites

Site (`hosted_sites_site`)

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`name`	CharField	
`slug`	SlugField	unique
`hostname`	CharField	null; unique
`status`	CharField	choices: active, disabled
`single_page_app`	BooleanField	
`kind`	CharField	choices: static, node
`application`	ForeignKey	null; → Application
`internet_hostname`	CharField	null; unique
`internet_published_by`	ForeignKey	null; → User
`internet_published_at`	DateTimeField	null
`internet_auto`	BooleanField	
`runtime_state`	CharField	
`runtime_log`	TextField	
`active_version`	ForeignKey	null; → SiteVersion
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

SiteVersion (`hosted_sites_siteversion`)

Field	Type	Notes
`id`	UUIDField	primary key
`site`	ForeignKey	→ Site
`number`	PositiveIntegerField	
`original_filename`	CharField	
`files`	JSONField	
`file_count`	PositiveIntegerField	
`total_bytes`	BigIntegerField	
`notes`	JSONField	
`manifest`	JSONField	
`uploaded_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

InternetGateway (`hosted_sites_internetgateway`)

The one internet gate of this installation (ADR-0020 decision 4): off unless the Super Administrator switches it on. The Cloudflare Tunnel token is kept encrypted and never shown again.

Field	Type	Notes
`id`	PositiveSmallIntegerField	primary key
`enabled`	BooleanField	
`token_encrypted`	BinaryField	null
`auto_domain`	CharField	
`changed_by`	ForeignKey	null; → User
`changed_at`	DateTimeField	null

imports

ImportJob (`imports_importjob`)

Field	Type	Notes
`id`	UUIDField	primary key
`file`	ForeignKey	→ FileObject
`table`	ForeignKey	→ DBTable
`encoding`	CharField	
`delimiter`	CharField	
`column_mapping`	JSONField	
`status`	CharField	choices: pending, running, completed, failed
`total_rows`	PositiveIntegerField	null
`imported_rows`	PositiveIntegerField	
`rejected_rows`	PositiveIntegerField	
`last_processed_row`	PositiveIntegerField	
`error_message`	TextField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	
`started_at`	DateTimeField	null
`completed_at`	DateTimeField	null

ImportJobError (`imports_importjoberror`)

Field	Type	Notes
`id`	UUIDField	primary key

Field	Type	Notes
`job`	ForeignKey	→ ImportJob
`row_number`	PositiveIntegerField	
`message`	CharField	
`raw_row`	JSONField	
`created_at`	DateTimeField	

licensing

LicenseState (`licensing_licensestate`)

Field	Type	Notes
`id`	BigAutoField	primary key
`trial_started_at`	DateTimeField	
`last_seen_at`	DateTimeField	
`serial`	CharField	
`activation_code`	TextField	
`activated_at`	DateTimeField	null
`activated_by`	ForeignKey	null; → User

movie_shop

MovieShop (`movie_shop_movieshop`)

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`project`	ForeignKey	→ Project
`name`	CharField	
`default_price`	PositiveIntegerField	
`agent_application`	OneToOneField	null; unique; → Application
`covers_bucket`	OneToOneField	null; unique; → Bucket
`agent_report`	JSONField	
`agent_seen_at`	DateTimeField	null
`portal_open`	BooleanField	
`welcome_points`	PositiveIntegerField	
`referral_points`	PositiveIntegerField	

Field	Type	Notes
`birthday_points`	PositiveIntegerField	
`every_nth_free`	PositiveSmallIntegerField	
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	

CategoryPrice (`movie_shop_categoryprice`)

The price of every title in one category (folder), unless a title sets its own.

Field	Type	Notes
`id`	BigAutoField	primary key
`shop`	ForeignKey	→ MovieShop
`category`	CharField	
`price`	PositiveIntegerField	

Library (`movie_shop_library`)

One folder on one of the shop PC's drives, as the media agent sees it (`` `D:\Movies` ``).

Field	Type	Notes
`id`	UUIDField	primary key
`shop`	ForeignKey	→ MovieShop
`name`	CharField	
`path`	CharField	
`kind`	CharField	choices: fixed, removable, network
`total_bytes`	BigIntegerField	null
`free_bytes`	BigIntegerField	null
`scan_requested_at`	DateTimeField	null
`scan_generation`	PositiveIntegerField	
`scanned_generation`	PositiveIntegerField	
`last_scan_at`	DateTimeField	null
`last_scan_message`	CharField	
`created_at`	DateTimeField	

Title (`movie_shop_title`)

Field	Type	Notes
`id`	UUIDField	primary key
`shop`	ForeignKey	→ MovieShop

Field	Type	Notes
`library`	ForeignKey	→ Library
`relative_path`	CharField	
`category`	CharField	
`title`	CharField	
`year`	PositiveSmallIntegerField	null
`kind`	CharField	choices: movie, series, music, album
`size_bytes`	BigIntegerField	
`episodes`	PositiveIntegerField	
`seen_generation`	PositiveIntegerField	
`cover`	ForeignKey	null; → FileObject
`cover_fingerprint`	CharField	
`cover_locked`	BooleanField	
`price`	PositiveIntegerField	null
`hidden`	BooleanField	
`description`	TextField	
`edited`	BooleanField	
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

PortalCustomer (`movie_shop_portalcustomer`)

A customer of one shop's portal (ADR-0022 decision 7). Not an IntraForge user: signs in only to this shop's portal, with a phone number and a password, and can do nothing else in IntraForge.

Field	Type	Notes
`id`	UUIDField	primary key
`shop`	ForeignKey	→ MovieShop
`name`	CharField	
`phone`	CharField	
`password`	CharField	
`birthday`	DateField	null
`referral_code`	CharField	
`referred_by`	ForeignKey	null; → PortalCustomer
`status`	CharField	choices: pending, approved, rejected, blocked
`pay_wallet_id`	UUIDField	null
`pay_username`	CharField	

Field	Type	Notes
`approved_at`	DateTimeField	null
`approved_by`	ForeignKey	null; → User
`birthday_points_year`	PositiveSmallIntegerField	null
`reset_requested_at`	DateTimeField	null
`created_at`	DateTimeField	

PortalSession (`movie_shop_portalsession`)

A portal sign-in: the cookie holds a random token, this table only its SHA-256.

Field	Type	Notes
`token_hash`	CharField	primary key
`customer`	ForeignKey	→ PortalCustomer
`created_at`	DateTimeField	
`expires_at`	DateTimeField	
`revoked_at`	DateTimeField	null

PointsEntry (`movie_shop_pointsentry`)

The points ledger: a customer's points are the sum of their entries. Never edited; a reversal is an entry.

Field	Type	Notes
`id`	BigAutoField	primary key
`customer`	ForeignKey	→ PortalCustomer
`delta`	IntegerField	
`reason`	CharField	choices: welcome, referral, birthday, redeem, returned, adjust
`sale`	ForeignKey	null; → Sale
`note`	CharField	
`created_at`	DateTimeField	

Sale (`movie_shop_sale`)

A counter sale: titles for one customer, paid with IntraForge Pay (or cash), then copied by the media agent onto the customer's drive. Nothing here holds money: Pay keeps the payment and its receipt (ADR-0022 decision 6).

Field	Type	Notes
`id`	UUIDField	primary key
`shop`	ForeignKey	→ MovieShop
`number`	CharField	unique

Field	Type	Notes
`status`	CharField	choices: ordered, awaiting_payment, paid, done, cancelled
`method`	CharField	choices: ifpay, cash
`customer_name`	CharField	
`total`	PositiveIntegerField	
`destination`	CharField	
`folder`	CharField	
`pay_request_id`	UUIDField	null
`pay_code`	TextField	
`pay_qr_svg`	TextField	
`pay_expires_at`	DateTimeField	null
`receipt_number`	CharField	
`paid_at`	DateTimeField	null
`customer`	ForeignKey	null; → PortalCustomer
`use_points`	BooleanField	
`points_used`	PositiveIntegerField	
`discount`	PositiveIntegerField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

SaleLine (`movie_shop_saleline`)

One title in a sale, as it was when sold (the catalogue may change later), and its copy onto the drive.

Field	Type	Notes
`id`	UUIDField	primary key
`sale`	ForeignKey	→ Sale
`title`	ForeignKey	null; → Title
`library`	ForeignKey	null; → Library
`title_name`	CharField	
`category`	CharField	
`relative_path`	CharField	
`kind`	CharField	
`price`	PositiveIntegerField	
`size_bytes`	BigIntegerField	
`free`	BooleanField	
`copy_status`	CharField	choices: waiting, copying, copied, failed

Field	Type	Notes
`copied_bytes`	BigIntegerField	
`copy_message`	CharField	
`copy_updated_at`	DateTimeField	null

Ad (`movie_shop_ad`)

An advertisement the shop shows on its own portal (ADR-0022 decision 8, phase 4): for the shop itself, one of its films, or another business the shop advertises for. Never shown anywhere else.

Field	Type	Notes
`id`	UUIDField	primary key
`shop`	ForeignKey	→ MovieShop
`title`	CharField	
`text`	CharField	
`advertiser`	CharField	
`film`	ForeignKey	null; → Title
`link`	URLField	
`picture`	ForeignKey	null; → FileObject
`starts_on`	DateField	null
`ends_on`	DateField	null
`active`	BooleanField	
`position`	PositiveIntegerField	
`views`	PositiveIntegerField	
`taps`	PositiveIntegerField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

PasswordResetCode (`movie_shop_passwordresetcode`)

A one-time code the owner gives a customer to choose a new portal password (shown once; only its hash is kept).

Field	Type	Notes
`id`	BigAutoField	primary key
`customer`	ForeignKey	→ PortalCustomer
`code_hash`	CharField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

Field	Type	Notes
`expires_at`	DateTimeField	
`used_at`	DateTimeField	null
`attempts`	PositiveSmallIntegerField	

oauth_provider

OAuthClient (`oauth_provider_oauthclient`)

Field	Type	Notes
`id`	UUIDField	primary key
`name`	CharField	
`client_id`	CharField	unique
`client_secret_hash`	CharField	
`application_type`	CharField	choices: confidential_web
`redirect_uris`	JSONField	
`post_logout_redirect_uris`	JSONField	
`allowed_origins`	JSONField	
`enabled`	BooleanField	
`requires_consent`	BooleanField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

OAuthAuthorizationCode (`oauth_provider_oauthauthorizationcode`)

Single-use, short-lived (services.py: `AUTHORIZATION_CODE_TTL_SECONDS`). The plaintext code is returned to the client exactly once, in the redirect — only its hash is stored, same rationale as ApplicationCredential/the code itself is never sensitive-data-bearing: it is a random lookup key, not a signed/encoded claim carrier.

Field	Type	Notes
`id`	UUIDField	primary key
`code_hash`	CharField	unique
`client`	ForeignKey	→ OAuthClient
`user`	ForeignKey	→ User
`redirect_uri`	CharField	
`scope`	CharField	
`code_challenge`	CharField	

Field	Type	Notes
`code_challenge_method`	CharField	
`nonce`	CharField	
`expires_at`	DateTimeField	
`used_at`	DateTimeField	null
`created_at`	DateTimeField	

OAuthAccessToken (`oauth_provider_oauthaccesstoken`)

Opaque, server-validated bearer token — the same shape as ApplicationCredential rather than a JWT, deliberately: this token is looked up and checked (expiry/revocation) on every use, exactly like the platform's existing bearer-token mechanism, and there is no resource server other than IntraForge's own `/oauth/userinfo/` that would need to verify it offline. The `id_token` (models.py has no row for it — see services.py `issue_id_token`) is the one token in this flow with a real offline-verification need (the spare-parts backend must check it without calling back into IntraForge), so it alone is a signed JWT. No refresh token is issued: the spare-parts backend exchanges the code once, reads `/oauth/userinfo/` (or just the ID token) immediately, and mints its OWN long-lived local session from that — this access token only needs to survive that one round trip, so a short, non-renewable lifetime (see ACCESS_TOKEN_TTL_SECONDS) is the deliberately simpler, smaller-attack- surface choice over adding refresh-token rotation/revocation.

Field	Type	Notes
`id`	UUIDField	primary key
`token_hash`	CharField	unique
`client`	ForeignKey	→ OAuthClient
`user`	ForeignKey	→ User
`scope`	CharField	
`expires_at`	DateTimeField	
`revoked_at`	DateTimeField	null
`created_at`	DateTimeField	

OAuthSigningKey (`oauth_provider_oauthsigningkey`)

RSA keypair used to sign `id_token` JWTs (RS256). The private key is Fernet-encrypted at rest (oauth_provider/crypto.py, same pattern as accounts/crypto.py's MFA-secret encryption, keyed by the existing CREDENTIAL_ENCRYPTION_KEY — no new secret to provision). `kid` is this row's own id, published in the JWKS alongside the public key so a client can select the right key after rotation; `active` marks the one key new tokens are signed with, while older, still-`active=False` keys remain published (until deliberately deleted) so a JWKS consumer can still verify a token issued just before a rotation.

Field	Type	Notes
`id`	UUIDField	primary key
`private_key_encrypted`	BinaryField	

Field	Type	Notes
`public_key_pem`	TextField	
`algorithm`	CharField	
`active`	BooleanField	
`created_at`	DateTimeField	

OAuthConsent (`oauth_provider_oauthconsent`)

A person's "Allow" for one application, and the scopes it covered. An application asking for more later asks again; deleting the row (or the client) withdraws it.

Field	Type	Notes
`id`	UUIDField	primary key
`client`	ForeignKey	→ OAuthClient
`user`	ForeignKey	→ User
`scope`	CharField	
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

org_units

OrganizationalUnit (`org_units_organizationalunit`)

A node in an organization's own structural hierarchy (branch, campus, department, warehouse, team, or any other label the organization chooses -- `unit_type` is a free-form descriptive label, never a hardcoded enum, so this stays a generic primitive rather than baking one industry's vocabulary into the platform, matching the App Platform's own "no industry-specific concept in the core" rule (see docs/implementation/APP_PLATFORM_PHASE4.md).

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`parent`	ForeignKey	null; → OrganizationalUnit
`unit_type`	CharField	
`name`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

UnitMembership (`org_units_unitmembership`)

A user's placement in a unit. Many-to-many by design (unlike organizations.Membership.team's single nullable FK) -- a person can sit in a department and a branch and a team simultaneously.

Field	Type	Notes
`id`	UUIDField	primary key
`unit`	ForeignKey	→ OrganizationalUnit
`user`	ForeignKey	→ User
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

organizations

Organization (`organizations_organization`)

Field	Type	Notes
`id`	UUIDField	primary key
`name`	CharField	
`slug`	SlugField	unique
`external_sharing_enabled`	BooleanField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

Team (`organizations_team`)

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`name`	CharField	
`created_at`	DateTimeField	

Membership (`organizations_membership`)

Field	Type	Notes
`id`	UUIDField	primary key
`user`	ForeignKey	→ User
`organization`	ForeignKey	→ Organization
`team`	ForeignKey	null; → Team
`status`	CharField	choices: invited, active, suspended
`invited_by`	ForeignKey	null; → User

Field	Type	Notes
`created_at`	DateTimeField	

permissions

Permission (`permissions_permission`)

Static, code-defined capability string (e.g. "storage.read"). The catalog is seeded from permissions.catalog.PERMISSIONS via the `seed_permissions` management command — never created ad hoc through the API (docs/security/PERMISSIONS.md Section 2).

Field	Type	Notes
`code`	CharField	primary key
`description`	CharField	

Role (`permissions_role`)

A named bundle of Permissions. `organization=None` marks one of the seeded system roles (docs/security/PERMISSIONS.md Section 3), usable by every organization and never deletable/renamable. An organization may later define its own custom roles (organization set, is_system=False) — not exercised yet in Phase 2.

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	null; → Organization
`slug`	SlugField	
`name`	CharField	
`is_system`	BooleanField	
`created_at`	DateTimeField	
`permissions`	ManyToManyField	→ Permission

RoleAssignment (`permissions_roleassignment`)

Grants a Role to a User, either scoped to one Organization or, for platform-wide operators, `organization=None`. A null organization is only valid when the assigned Role includes `system.admin` (enforced in permissions.services.assign_role, not at the DB layer, since "which permissions a role has" is data, not schema).

Field	Type	Notes
`id`	UUIDField	primary key
`user`	ForeignKey	→ User
`role`	ForeignKey	→ Role

Field	Type	Notes
`organization`	ForeignKey	null; → Organization
`granted_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

ResourceGrant (`permissions_resourcegrant`)

Grants exactly one Permission on exactly one resource to one User, with an optional expiry. Used for internal sharing (Phase 9) and for restricting an Application's scope to specific resources (Phase 7) — the same mechanism, per ADR-0008. `resource_type` is a free-form label until concrete resource types exist (Phase 3+); nothing in Phase 2 grants against real resources yet.

Field	Type	Notes
`id`	UUIDField	primary key
`user`	ForeignKey	→ User
`permission`	ForeignKey	→ Permission
`organization`	ForeignKey	→ Organization
`resource_type`	CharField	
`resource_id`	UUIDField	
`granted_by`	ForeignKey	null; → User
`expires_at`	DateTimeField	null
`created_at`	DateTimeField	

service_apps

ServiceApp (`service_apps_serviceapp`)

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`project`	ForeignKey	→ Project
`deployment`	CharField	unique
`app_type`	CharField	choices: spare_parts, intraforge_pay, pharmacy
`display_name`	CharField	
`config`	JSONField	
`enabled`	BooleanField	
`removed_at`	DateTimeField	null
`schema_name`	CharField	unique
`bucket`	OneToOneField	unique; → Bucket

Field	Type	Notes
`reported_version`	CharField	
`last_seen_at`	DateTimeField	null
`created_by`	ForeignKey	→ User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

ServiceAppSession (`service_apps_serviceappsession`)

A browser session for one service application, minted by "Open" in IntraForge.

Field	Type	Notes
`id`	BigAutoField	primary key
`token_hash`	CharField	unique
`service_app`	ForeignKey	→ ServiceApp
`user`	ForeignKey	→ User
`django_session_key`	CharField	
`created_at`	DateTimeField	
`expires_at`	DateTimeField	
`last_used_at`	DateTimeField	null
`revoked_at`	DateTimeField	null

ServiceAppAuditReceipt (`service_apps_serviceappauditreceipt`)

Makes audit relay idempotent: the service sends each event at least once, identified by the id it gave the event in its own transactional log; IntraForge records it exactly once.

Field	Type	Notes
`event_id`	CharField	primary key
`service_app`	ForeignKey	→ ServiceApp
`audit_event`	ForeignKey	→ AuditEvent
`received_at`	DateTimeField	

sharing

ShareGrant (`sharing_sharegrant`)

Records that a resource was shared with a principal (User/Team/ Organization) at a given level. This is the human-facing "who shared what with whom" record — it is not a second enforcement path. Enforcement is entirely the existing `permissions.ResourceGrant` / `has_permission` mechanism (docs/architecture/DATA_MODEL.md Section 3.8; permissions/models.py's `ResourceGrant` docstring: "Used

for internal sharing (Phase 9) and for restricting an Application's scope to specific resources (Phase 7) — the same mechanism, per ADR-0008"). `sharing/services.py` creates one `ResourceGrant` per (target user, permission code) implied by `level` when a `ShareGrant` is created, and removes them on revoke.

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`resource_type`	CharField	
`resource_id`	UUIDField	
`principal_type`	CharField	choices: user, team, organization
`user`	ForeignKey	null; → User
`team`	ForeignKey	null; → Team
`level`	CharField	choices: read, write, admin, sell, receive, adjust, transfer
`expires_at`	DateTimeField	null
`revoked_at`	DateTimeField	null
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	

storage

Bucket (`storage_bucket`)

Field	Type	Notes
`id`	UUIDField	primary key
`project`	ForeignKey	→ Project
`environment`	OneToOneField	null; unique; → Environment
`name`	CharField	
`versioning_enabled`	BooleanField	
`trash_days`	PositiveIntegerField	
`version_days`	PositiveIntegerField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

Folder (`storage_folder`)

Virtual hierarchy stored as metadata rows (adjacency list) — never a real filesystem directory (docs/architecture/DATA_MODEL.md Section 3.4).

Field	Type	Notes
`id`	UUIDField	primary key
`bucket`	ForeignKey	→ Bucket
`parent`	ForeignKey	null; → Folder
`name`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

FileObject (`storage_fileobject`)

Field	Type	Notes
`id`	UUIDField	primary key
`bucket`	ForeignKey	→ Bucket
`folder`	ForeignKey	null; → Folder
`object_key`	CharField	unique
`original_filename`	CharField	
`display_filename`	CharField	
`mime_type`	CharField	
`size`	BigIntegerField	
`checksum_sha256`	CharField	
`storage_backend`	CharField	
`status`	CharField	choices: active, deleted, quarantined
`creator`	ForeignKey	→ User
`created_at`	DateTimeField	
`updated_at`	DateTimeField	

FileVersion (`storage_fileversion`)

Append-only version history, populated when a file is overwritten in a bucket with versioning enabled (docs/architecture/DATA_MODEL.md Section 3.4).

Field	Type	Notes
`id`	UUIDField	primary key
`file`	ForeignKey	→ FileObject
`object_key`	CharField	unique
`size`	BigIntegerField	
`checksum_sha256`	CharField	
`mime_type`	CharField	

Field	Type	Notes
`quarantined`	BooleanField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

FileShareLink (`storage_filesharelink`)

A link that lets anyone who has it download one file until it expires (at most 30 days), is revoked, or reaches its download limit. Only a hash of the link's token is kept; the link is shown once.

Field	Type	Notes
`id`	UUIDField	primary key
`file`	ForeignKey	→ FileObject
`token_hash`	CharField	unique
`expires_at`	DateTimeField	
`max_downloads`	PositiveIntegerField	null
`downloads`	PositiveIntegerField	
`created_by`	ForeignKey	null; → User
`created_at`	DateTimeField	
`revoked_at`	DateTimeField	null

system

BackupRecord (`system_backuprecord`)

One `pg_dump` attempt against a control-plane or tenant PostgreSQL connection (system/backups.py). `verified_restorable` is never set True by the backup itself — only a subsequent, independent restore- and-validate pass against an isolated database sets it (docs/operations/BACKUP_RESTORE.md Section 6/7: "a backup strategy is not considered complete until restoration has actually been tested").

Field	Type	Notes
`id`	UUIDField	primary key
`backup_type`	CharField	choices: control_db, tenant_db, object_storage, configuration
`status`	CharField	choices: running, success, failed
`file_path`	CharField	
`size_bytes`	BigIntegerField	null
`error_message`	TextField	
`started_at`	DateTimeField	
`completed_at`	DateTimeField	null
`verified_restorable`	BooleanField	

Field	Type	Notes
`verified_at`	DateTimeField	null
`verification_error`	TextField	
`restored_at`	DateTimeField	null
`restore_error`	TextField	

HealthSample (`system_healthsample`)

One run of the health checks (system.health.record, every 5 minutes): which checks passed. Kept HEALTH_HISTORY_DAYS days.

Field	Type	Notes
`id`	BigAutoField	primary key
`at`	DateTimeField	
`checks`	JSONField	

HealthAlert (`system_healthalert`)

A check that failed twice in a row; closed by itself when the check passes again.

Field	Type	Notes
`id`	BigAutoField	primary key
`check_name`	CharField	
`detail`	CharField	
`opened_at`	DateTimeField	
`closed_at`	DateTimeField	null

workspaces

Workspace (`workspaces_workspace`)

Field	Type	Notes
`id`	UUIDField	primary key
`organization`	ForeignKey	→ Organization
`name`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

Project (`workspaces_project`)

Field	Type	Notes
`id`	UUIDField	primary key
`workspace`	ForeignKey	→ Workspace
`name`	CharField	
`created_at`	DateTimeField	
`created_by`	ForeignKey	→ User

Reference C — the modules

Every source file, with the first paragraph of its own description (docstring or leading comment), generated by `docs/guides/tools/gen_modules.py`. A dash means the file has no description yet — read the code.

Backend apps (`apps/backend/<app>/`)

accounts

File	What it is
`crypto.py`	Encryption for MFA (TOTP) secrets at rest — same approach as `databases/crypto.py` (Fernet, keyed by `CREDENTIAL_ENCRYPTION_KEY`), but kept as its own small module rather than a shared import: `accounts` is the most foundational app (every other app depends on it via `settings.AUTH_USER_MODEL`), and having it import from a later, feature- specific app like `databases` would invert this codebase's
`management/commands/bootstrap_super_administrator.py`	—
`management/commands/passkey_reset.py`	—
`middleware.py`	After an administrator reset someone's password, that person may do nothing through the API until they choose their own (accounts.services.reset_password sets must_change_password): only seeing who they are, changing the password, signing out and the CSRF cookie stay open.
`models.py`	—
`passkeys.py`	Passkeys (WebAuthn), ADR-0019. Kept apart from the HTTP layer like accounts/services.py.
`serializers.py`	—
`services.py`	MFA (TOTP) lifecycle (Phase 10, docs/architecture/ROADMAP.md). Kept separate from the HTTP layer (accounts/views.py) so enrollment/ confirmation/verification are testable without going through session machinery.
`totp.py`	RFC 6238 TOTP (the algorithm behind Google Authenticator / Authy / 1Password's TOTP support), implemented directly against the standard library rather than adding a third-party dependency for something this small and security-critical to get right ourselves: SHA-1/HMAC, base32, and struct packing are all `hashlib` / `hmac` / `base64` / `struct` — nothing here needs an external package (Section 24 of the
`urls.py`	—
`views.py`	—

analytics

File	What it is
`data.py`	Fetches column data for analysis directly from the tenant Postgres connection — safely quoted the same way `databases/rows.py` already does (`psycopg.sql.Identifier` for identifiers, never string interpolation), and always row-capped before a single byte reaches Python/numpy/scipy (Section 20 of the master prompt: a statistics engine must not become a denial-of-service vector).
`models.py`	—
`operations.py`	The fixed, versioned registry of analytics operations (Section 27 of the master prompt: "expose only approved analytics operations through validated server-side APIs" — never arbitrary code, not even arbitrary SQL). Each function takes `(table, params)` and returns a JSON-safe dict. Statistical (as opposed to purely descriptive) operations always include `method` , `sample_size` , `assumptions` , and
`profiling.py`	Automatic dataset profiling — a "Data Quality Report" for a whole table in one call (Section 23 of the master prompt), computed from a single capped fetch of every column rather than one query per column.
`serializers.py`	—
`services.py`	Permission checks, audit events, and row/column caps around the analytics operation registry (operations.py) and dashboards. Running analysis or viewing a dashboard requires `database.read` on the underlying table (the same read capability the data explorer already uses — analysis doesn't expose anything reading the raw rows wouldn't); creating/editing a dashboard requires `dataset.analyze` , since
`urls.py`	—
`views.py`	—

app_platform

File	What it is
`access.py`	—
`attachment_views.py`	Attachment endpoints; see attachments.py for authorization/validation.
`attachments.py`	Attach an already-uploaded `storage.FileObject` to a record.
`built_in_apps.py`	Built-in business applications: the customer-facing catalog and one-click install.
`built_in_views.py`	Built-in application gallery, one-click install and setup retry (see built_in_apps.py).
`definitions.py`	Strict, bounded, non-executable template format shared by API and services.
`delivery_transport.py`	Serialized, bounded, durably scheduled HTTP delivery attempts.
`instances.py`	Independent installation and serialized metadata edits; no tenant DDL.

File	What it is
`inventory_authority.py`	Business-operation authority over an installed application's stock (ADR-0017).
`inventory_operations.py`	Controlled business operations over an installed inventory application: SELL and RECEIVE (ADR-0017).
`management/commands/seed_reference_app.py`	`python manage.py seed_reference_app --template <key> --organization <id-or-slug> --actor-email <email> [--project <id>] [--provision]`
`management/commands/seed_spare_parts_reference_app.py`	`python manage.py seed_spare_parts_reference_app --organization <id-or-slug> --actor-email <email> [--project <id>] [--provision]`
`management/commands/upgrade_operation_journals.py`	Operator-only, additive upgrade of existing managed runtime schemas.
`models.py`	Control-plane application definitions; never business-record storage.
`operation_journal.py`	Tenant-atomic record events with replayable control-plane publication (ADR-0016).
`outbound_http.py`	Bounded webhook POST transport: validated numeric destination, no redirects/proxy.
`provisioning.py`	Durable reservation, replay and publication of a generated app database.
`quantity_ledger.py`	ADR-0016's closing half: an opt-in, generic integer quantity ledger.
`record_views.py`	Record CRUD over a provisioned app runtime; see records.py for the translation/authorization/validation this thin view layer delegates to.
`records.py`	Typed record CRUD over a provisioned app runtime.
`reference_templates/_helpers.py`	Shared, pure, DB-free builder helpers for every reference-template module in this package (docs/implementation/APP_PLATFORM_PHASE4.md, APP_PLATFORM_PHASE8.md). Each template module produces a `DefinitionInput`-shaped dict using only these three helpers plus Phases 1-3/6's existing field/relationship vocabulary -- no industry-specific concept is ever added to `app_platform` itself.
`reference_templates/assets.py`	Phase 8 -- Assets (fixed-asset management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "Assets -- Lifecycle & Depreciation" AppTemplate draft: 8 generic models covering acquisition, assignment, maintenance, depreciation, and disposal, built entirely from Phases 1-3/6-7's existing primitives, includ
`reference_templates/crm.py`	Phase 8 -- CRM (customer relationship management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "CRM -- Sales Pipeline" AppTemplate draft: 8 generic models covering accounts, contacts, leads, campaigns, and the opportunity pipeline, built entirely from Phases 1-3's existing primitives. No industry-sp

File	What it is
`reference_templates/documents.py`	Phase 8 -- Documents (document management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "Documents -- Library & Review" AppTemplate draft: 6 generic models covering folders, categorization, versioning, tagging, and review, built entirely from Phases 1-3's existing primitives, including a self-refere
`reference_templates/hr.py`	Phase 8 -- HR (human resources) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "HR -- Workforce & Payroll" AppTemplate draft: 9 generic models covering organizational structure, leave, time, payroll, and performance, built entirely from Phases 1-3/6-7's existing primitives -- including a self-referenc
`reference_templates/pharmacy.py`	Pharmacy -- Medicines, Stock & Sales: the Spare Parts reference application (spare_parts.py) in its pharmacy edition. Same models, relationships, constraints and quantity ledger -- so stock, receiving and selling run through exactly the qualified engine (ADR-0016, ADR-0017) -- with the medicine facts a pharmacy needs on the product: generic name, pack, manufacturer, origin, whether a prescription
`reference_templates/procurement.py`	Phase 8 -- Procurement reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "Procurement -- Requisition to Pay" AppTemplate draft: 9 generic models covering requisitions, purchase orders, receiving, vendor invoicing, and approval, built entirely from Phases 1-3's existing primitives. No industry-specific co
`reference_templates/projects.py`	Phase 8 -- Projects (project management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "Projects -- Delivery & Time Tracking" AppTemplate draft: 8 generic models covering projects, milestones, tasks, assignments, time tracking, and risk, built entirely from Phases 1-3's existing primitives. No indust
`reference_templates/rentals.py`	Rentals -- items and bookings. A pure, DB-free builder for a shop that hires things out (tools, party equipment, cameras, cars): the items and their daily price, customers, bookings with their dates, deposit and condition on return, and payments. Built only from app_platform's generic primitives.
`reference_templates/repair_shop.py`	Repair Shop -- job cards. A pure, DB-free builder for a phone, computer or appliance repair counter: customers, the devices they bring, a job card per repair (fault, status, estimate, promised date, technician), the parts and labour on it, and payments. Built only from app_platform's generic primitives; no repair-specific concept is added to app_platform itself.
`reference_templates/school.py`	Phase 8 -- School (education management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "School -- Enrollment & Records" AppTemplate draft: 11 generic models covering academic structure, enrollment, attendance, grading, and billing, built entirely from Phases 1-3/6's existing primitives. No industry-
`reference_templates/spare_parts.py`	Phase 4 -- Inventory and Sales reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE4.md). A pure, DB-free builder for the "Spare Parts -- Inventory & Sales" AppTemplate draft: 13 generic models covering inventory, purchasing, and sales, built entirely from Phases 1-3's existing primitives (scalar fields, many_to_one/many_to_many relationships,
`reference_templates/warehouses.py`	Phase 8 -- Warehouses (warehouse / logistics management) reference application (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE8.md). A pure, DB-free builder for the "Warehouses -- Bin-Level Logistics" AppTemplate draft: 10 generic models covering warehouse/zone/bin structure, bin-level stock, and inbound/outbound/transfer movement, built entirely from Phases 1-

File	What it is
`runtime_build.py`	Validated tenant/catalog construction inside provisioning's transactions.
`runtime_plan.py`	Read-only, deterministic relational plan for an installed application.
`runtime_views.py`	Runtime preflight, reservation and asynchronous provisioning status.
`schema_evolution.py`	Additive-only live schema changes against an already-provisioned, possibly populated runtime (Phase 3 step 3).
`serializers.py`	—
`tasks.py`	Provisioning and at-least-once HTTP delivery with durable retry scheduling.
`templates.py`	Template services: validated drafts and serialized immutable publication.
`transactions.py`	Atomic multi-record business operations (ADR-0016; Local Edition P0: no partial business transaction commit).
`unit_scope.py`	Phase 7 Step 3 ("unit scope") service layer: grant/revoke a UnitScopedGrant, and resolve which units (if any) an actor is restricted to for a given model/capability -- the one function records.py actually calls on every record operation.
`upgrade_views.py`	Phase 9 -- template versioning and upgrade endpoints.
`upgrades.py`	Phase 9 -- template versioning and upgrades (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE9.md).
`urls.py`	—
`views.py`	Small metadata endpoints. Mutations and permission checks stay in services.
`webhooks.py`	Phase 10 -- reliable webhooks (docs/implementation/APP_PLATFORM_ROADMAP.md, docs/implementation/APP_PLATFORM_PHASE10.md). Closes the one gap `docs/EXTERNAL_APP_API_CONTRACT.md` previously listed outright as **NOT YET IMPLEMENTED**: "Webhooks or any push/event-notification mechanism out of App Platform toward an external application."
`workflows.py`	Phase 5 -- workflow transition application (docs/implementation/APP_PLATFORM_PHASE5.md): the concurrency-safe engine that applies one declared WorkflowTransition to one record. Mirrors provisioning.py's own reserve-then-execute shape almost exactly -- a durable receipt row (WorkflowTransitionAttempt) whose `completed_at` is the single source of truth for "did this already happen," and the same uni

applications

File	What it is
`authentication.py`	Service-account authentication for the API (Section 14 of the master prompt). A valid `Authorization: Bearer pdc_sk_...` resolves to the credential's ServiceAccount's `identity_user` — from that point on, every existing permission check, tenant-scoped lookup, and audit call treats it exactly like a human user's session, because it *is* the same `User` model (see applications/models.py:ServiceAccou

File	What it is
`middleware.py`	Records each request an application makes with its bearer credential (ApiRequestLog), after the response. DRF authenticates inside the view and sets request.auth on the underlying request, so a request with no ApplicationCredential (every person's own session) is never recorded. Recording never changes or fails the response: a write that fails is logged and the response goes out as it was.
`models.py`	—
`serializers.py`	—
`services.py`	Application/service-account lifecycle: register an Application (creates its ServiceAccount and the User row that identity resolves to), issue/ rotate/revoke credentials, and resolve a bearer token back to a ServiceAccount at request time (applications/authentication.py).
`tasks.py`	—
`urls.py`	—
`views.py`	—

audit

File	What it is
`models.py`	—
`serializers.py`	—
`services.py`	—
`urls.py`	—
`views.py`	—

catalogue_storage

File	What it is
`management/commands/import_diagram_images.py`	`python manage.py import_diagram_images --dir /path/to/drawings --license-confirmed`
`management/commands/import_oem_catalogue.py`	`python manage.py import_oem_catalogue --csv parts.csv --images-dir images/ --provider toyota-epc --manufacturer Toyota`
`management/commands/rotate_catalogue_credential.py`	—

File	What it is
`models.py`	The minimum private catalogue this project actually needs (see the "Simplify the Toyota/OEM integration" decision this closes — docs/CATALOGUE_STORAGE.md): OEM part identity, one diagram image shared by every part drawn on it, and the join between them (callout + optional hotspot rect). Deliberately NOT a vehicle/EPC clone — no Make/Model/ Generation/Variant hierarchy, no fitment matrix. A part's
`serializers.py`	—
`services.py`	Business logic shared by the seed migration, the CSV importer management command, and (read-only) the API views — one place that knows how to normalize, deduplicate, and link, so all three stay consistent.
`urls.py`	—
`views.py`	—

databases

File	What it is
`connections.py`	Service layer for `ConnectedDatabase` (Phase 8, ADR-0009): register/test/ browse/delete an externally-hosted PostgreSQL database in connected mode. Deliberately separate from `databases/services.py` (which owns the TenantDatabase DDL pipeline) — this module never issues DDL/DML against anything the platform owns; every SQL statement it triggers runs against the *external* system through `databases
`connectors.py`	Shared external-database connector interface (ADR-0009): connection testing and read-only schema/data access for a `ConnectedDatabase`, kept entirely separate from the TenantDatabase DDL/DML code path (databases/ddl.py, databases/rows.py) since a connected-mode source is never treated as platform-native storage — nothing here ever writes to the external database.
`crypto.py`	Encryption for ConnectedDatabase credentials at rest (Section 15 of the master prompt; ADR-0009). Uses CREDENTIAL_ENCRYPTION_KEY, a key dedicated to this purpose and distinct from Django's SECRET_KEY, so rotating one never silently affects the other.
`ddl.py`	Safe DDL construction — layer 2 of the two-layer defense Section 5 of the master prompt requires ("1. strict identifier validation, 2. safe identifier quoting"). Every identifier here has already passed databases/identifiers.py validation; this module is responsible for quoting identifiers (`psycopg.sql.Identifier`) and safely embedding constant values (`psycopg.sql.Literal`) — never plain string
`formats.py`	Shared parsing constants for turning raw text/JSON input into a DBColumn.DataType-typed Python value — used by CSV import (imports/inspect.py, imports/services.py) and the data explorer's row API (databases/values.py) alike, so the two paths can't silently drift apart on what counts as a valid date/datetime/boolean.
`identifiers.py`	Strict identifier validation — layer 1 of the two-layer defense Section 5 of the master prompt requires for anything that dynamically creates schemas/tables ("1. strict identifier validation, 2. safe identifier quoting"). Every schema/table/column name that ends up in dynamically constructed DDL is validated here *before* being passed to `psycopg.sql.Identifier` for quoting (layer 2, in databases/
`models.py`	—
`normalization.py`	The one rule that turns typed identifiers into a comparable search key.

File	What it is
`rows.py`	The data explorer's row-level query/mutation layer (Section 12 of the master prompt): server-side pagination, sorting, filtering, and search against a real tenant table via raw, safely-quoted SQL — Django's ORM can't model these tables since their schema is only known at runtime (Phase 4). Table/column identifiers here always come from already- validated catalog rows (`databases/identifiers.py` va
`serializers.py`	—
`services.py`	The schema-change service layer. Every operation here follows the same pipeline required by Section 9 of the master prompt: validate permission, validate the requested schema, start a transaction, safely perform the operation, save metadata, generate an audit event, gracefully roll back on failure.
`urls.py`	—
`values.py`	Validates and converts a JSON-native value (as received in a row insert/update request body) against a column's `DBColumn.DataType` — the data-explorer counterpart to `imports/services.py` 's `_convert_value` , which does the same job starting from raw CSV strings instead. Different input shapes (JSON already has native int/bool/dict; CSV is always text), so not fully shared, but both draw date/date
`views.py`	—

environments

File	What it is
`crypto.py`	Encryption for EnvironmentSecret values and EnvironmentWebhook signing secrets at rest -- same approach as `accounts/crypto.py` / `databases/ crypto.py` (Fernet, keyed by `CREDENTIAL_ENCRYPTION_KEY`), kept as its own small module rather than a shared import, matching this codebase's established per-app convention (see accounts/crypto.py's own docstring for why: a handful of duplicated lines beats a
`models.py`	Per-application Environment management: Organization -> Application -> Environment, per docs/architecture/ROADMAP.md's Environment Management subsystem entry. An Environment is the isolation boundary a Development credential must never cross into Production — `is_production_tier` (not `environment_type` , a free-form label) is the field every isolation and RBAC check in this app and in databases/st
`serializers.py`	—
`services.py`	Environment lifecycle: create/update/clone/disable/delete, secrets, variables, webhooks, environment-scoped credential issuance (reusing applications.services -- never a second credential mechanism), and the runtime isolation check databases/storage call before returning row or file data to a credential-authenticated request.
`urls.py`	—
`views.py`	—

exports

File	What it is
`budgets.py`	Resource limits for portable packages, before decompressing any member.
`builder.py`	Builds a .icp package for one Organization: its workspace/project tree, tenant databases (schema + row data), object storage (bucket/folder/file structure + real file bytes), and membership/role-assignment metadata.
`container.py`	The .icp file's outer container format (Section 19 of the master prompt). Structure, all big-endian:
`crypto.py`	Encrypts an export/restore passphrase for the short hop through Celery (Valkey broker + task-received/task-succeeded log lines, both of which would otherwise carry it in plaintext — Celery logs task args at INFO by default). Reuses CREDENTIAL_ENCRYPTION_KEY, the same key `databases/crypto.py` uses for ConnectedDatabase credentials, since it's already the designated "encrypt an operator-facing secr
`manifest.py`	The .icp manifest: a versioned, machine-readable description of a portable export package's contents (Section 13 of the master prompt). Kept as a plain dict (not a Django model) — it lives inside the package itself, travels to other installations, and must be readable by a future version of this code that may not share this version's models.
`models.py`	—
`preparation.py`	Repeatable external preparation, before any restore publication locks.
`recovery.py`	Durable portable restore orchestration (not production backup restore).
`restorer.py`	Restores a .icp package into a brand-new Organization (the only target this first implementation supports — restoring *into* an existing organization needs a conflict-resolution policy this doesn't have yet, tracked as an open item). Schema and rows are rebuilt exclusively through the same validated service functions the live database builder and file-upload pipeline already use (`databases.servic
`serializers.py`	—
`services.py`	Orchestration for export/restore jobs: permission checks, audit events, and moving the .icp package's bytes to/from object storage — kept separate from builder.py/restorer.py, which know nothing about jobs, permissions, or storage keys and are reusable on their own.
`tasks.py`	—
`urls.py`	—
`views.py`	—

imports

File	What it is
`inspect.py`	CSV inspection for the import preview step (Section 11 of the master prompt: Upload -> Inspect -> Detect encoding/delimiter -> Parse headers -> Sample rows -> Infer candidate types -> Present preview). Operates on a bounded sample only — never reads a whole (potentially multi-gigabyte) file for a preview.
`models.py`	—
`serializers.py`	—
`services.py`	The CSV import pipeline's execution layer. `start_import` (called from the view, synchronous, cheap) validates and creates the `ImportJob`, then enqueues the actual work. `run_import` (called from the Celery task) does the real streamed, chunked bulk insert against the tenant connection.
`tasks.py`	—
`urls.py`	—
`views.py`	—

licensing

File	What it is
`codes.py`	Serial numbers, PC codes and activation codes (ADR-0023). Pure Python with `cryptography` only -- no Django -- so the vendor's licence tool (scripts/licensing/intraforge-license.py) uses exactly the same rules as IntraForge.
`middleware.py`	Read-only without a licence (ADR-0023): after the trial, without a valid licence, a request that would change something is refused -- except signing in and out (IntraForge's own, and answering an application's consent screen), activating the licence, and the internal service API.
`models.py`	This installation's licence (ADR-0023): one row.
`services.py`	The licence (ADR-0023): a 30-day trial, then a serial number and an activation code for this PC.
`urls.py`	—
`views.py`	—

movie_shop

File	What it is
`ads.py`	Ads on a shop's customer portal (ADR-0022 decision 8, phase 4).
`ads_views.py`	Ads API (ADR-0022 phase 4): the owner's side (movies.manage, a signed-in session) and the portal's side (the shop's open portal; no sign-in needed to see the shop's ads).

File	What it is
`models.py`	Movie Shop (ADR-0022): a catalogue of the shop's own video library. Only metadata lives here -- the films stay on the shop's drives and the media agent (a Windows client of this API) reads and copies them; covers are ordinary IntraForge files in the shop's `movie-covers` bucket.
`portal.py`	The customer portal (ADR-0022 phase 3): a shop's customers sign up on the shop Wi-Fi, browse the catalogue, place orders the counter charges, and collect Movie Shop points.
`portal_views.py`	The customer portal's API (ADR-0022 phase 3), at /api/v1/portal/<shop>/.
`sales.py`	Counter sales (ADR-0022 phase 2): a cart of titles for one customer, paid through IntraForge Pay (the existing Pay bridge, ADR-0018) or in cash, then copied by the media agent onto the customer's drive.
`scanning.py`	What the media agent may tell IntraForge, checked here whatever the agent did (ADR-0022 decision 5): drive reports, library paths and scanned titles. Nothing here trusts a value because it came from our own agent.
`services.py`	Movie Shop business logic (ADR-0022). Views check who may do what; everything that changes data happens here.
`urls.py`	—
`views.py`	Movie Shop API (ADR-0022). Owner endpoints take only a signed-in person's session (never an application credential); the media agent's endpoints take only that shop's own agent credential.

oauth_provider

File	What it is
`admin_urls.py`	Versioned admin API for registering/managing OAuth clients — included under /api/v1/ from config/api_urls.py, unlike urls.py's protocol endpoints (see that module's own docstring).
`crypto.py`	Encryption for the OIDC signing key's private half — same approach as accounts/crypto.py (Fernet, keyed by CREDENTIAL_ENCRYPTION_KEY), kept as its own small module for the same reason accounts/crypto.py gives: avoiding a cross-app import for ~10 lines. Hashing helpers here mirror ApplicationCredential's convention (SHA-256 hex of a high-entropy, server-generated secret — not a slow password hasher
`models.py`	OAuth 2.1-style Authorization Code + PKCE, with an OpenID Connect layer on top — the identity-provider surface for *external* applications (e.g. the Harmoney Spare Parts website) that need "Sign in with IntraForge", as distinct from two existing, deliberately different mechanisms:
`serializers.py`	—
`services.py`	Business logic for the Authorization Code + PKCE (+ OIDC) flow. Views stay thin — every rule the OAuth/OIDC spec and this project's own security requirements impose lives here, so it is exercised identically whether called from `views.py` or a test.
`urls.py`	Protocol endpoints — mounted at the application root in config/urls.py (not under /api/v1/), because /.well-known/openid-configuration's path is spec-mandated to be exactly that, and the /oauth/* endpoints sit alongside it by convention rather than inside the versioned resource API. See infrastructure/proxy/Caddyfile's `@backend` matcher, which must route these paths to this service too.
`views.py`	—

org_units

File	What it is
`models.py`	—
`serializers.py`	—
`services.py`	—
`urls.py`	—
`views.py`	—

organizations

File	What it is
`models.py`	—
`serializers.py`	—
`services.py`	—
`urls.py`	—
`views.py`	—

permissions

File	What it is
`catalog.py`	The permission catalog and system role definitions, straight from docs/security/PERMISSIONS.md Sections 2 and 3. This module is the single source of truth `seed_permissions` loads into the database — new permissions/roles are added here (and in the doc), never invented ad hoc in a view or migration.
`management/commands/seed_permissions.py`	—
`models.py`	—
`services.py`	The single shared authorization service (ADR-0008). Every permission check in the codebase — view, service, background task — goes through `has_permission`, never an inline role-name check.

service_apps

File	What it is
`app_pay.py`	IntraForge Pay for applications (ADR-0020 decision 2): an Application credential -- for example a hosted Node.js app's IF_API_TOKEN -- asks this organization's IntraForge Pay to take a payment, through the same Pay bridge the service applications use (pay_bridge.py). IntraForge picks the Pay (the application's own organization), and the payment is taken by the application's service account, which
`authentication.py`	Authentication for the internal service-application API (ADR-0018).

File	What it is
`deployments.py`	The service deployments that exist in this installation (settings.SERVICE_APP_DEPLOYMENTS).
`internal_views.py`	Internal API for service applications (ADR-0018). Reachable only on the internal network (the proxy refuses /api/v1/internal/), authenticated with the deployment's service token, and scoped to that deployment's own instance: there is no way to name another instance.
`management/commands/list_service_apps.py`	—
`management/commands/sync_service_apps.py`	—
`models.py`	Separately deployed service applications (ADR-0018).
`pay_bridge.py`	Taking an IntraForge Pay payment on another application's behalf (ADR-0018, amended 2026-10-24).
`provisioning.py`	Database provisioning for service applications: one login role per deployment, one schema per installed instance, in the tenant PostgreSQL database (ADR-0005, ADR-0018).
`services.py`	Service-application lifecycle and the platform services a service application uses.
`sessions.py`	Browser sessions for service applications (ADR-0018 §3).
`signals.py`	—
`types.py`	What IntraForge knows about each service application type.
`urls.py`	—
`views.py`	Public API: the Apps registry, settings, removal and "Open" (human sessions only).

sharing

File	What it is
`models.py`	—
`serializers.py`	—
`services.py`	Internal resource sharing (Phase 9, ADR-0008): grants a User/Team/ Organization read/write/admin-level access to exactly one resource by creating the underlying `ResourceGrant`(s) that `permissions.services.has_permission` already knows how to check — the same mechanism Phase 7 uses to scope an Application's access to one resource (permissions/models.py's `ResourceGrant` docstring). `ShareGrant`
`urls.py`	—
`views.py`	—

sites

File	What it is
`archive.py`	Checks an uploaded website zip before anything is stored (ADR-0019 decision 3). Nothing here trusts the zip: names, types, sizes and counts are all validated, and the result is a plan of exactly which entries become which site paths. Reading an entry afterwards is bounded by its declared size (zipfile stops at `file_size`).
`internet.py`	The internet gate (ADR-0020 decision 4). Off by default. Only the Super Administrator (the platform-wide `sites.internet.manage` capability) can switch it on -- with their password, and only with two-factor sign-in on their account -- and publish or revoke a site under a public name. The Cloudflare Tunnel connector runs only while the gate is on and reaches only the proxy's internet listener, wh
`models.py`	Hosted websites (ADR-0019): an organization uploads a finished static website as a zip and IntraForge serves it on its own port. A `SiteVersion` is one uploaded zip, immutable once stored; its files live in object storage under `sites/<site id>/<version id>/` and `files` is the manifest of exactly what may be served.
`runner.py`	The backend's side of the app runner (ADR-0020): a token-authenticated HTTP client for its fixed operations. The backend never talks to Docker; it hands the runner a server app's files and metadata and reads back state.
`serializers.py`	—
`serve.py`	The public side of hosted sites (ADR-0019 decision 4): `/_sites/<slug>/<path>` on the backend, reached only through the proxy's sites port, which rewrites `https://<PC>:8445/<slug>/...` to it and strips cookies and credentials. Only paths in the active version's manifest are ever served; the request's user is never consulted.
`serve_urls.py`	—
`services.py`	Hosted-site lifecycle (ADR-0019): create a site from a zip, upload further versions, switch the active version, take a site offline, delete versions and sites. Every change records an audit event. Files are written to object storage before the version row exists and removed again if anything fails, so a version row always has its files.
`urls.py`	—
`views.py`	—

storage

File	What it is
`backends.py`	Thin, provider-agnostic wrapper around boto3's S3 client — MinIO locally, AWS S3 or another S3-compatible provider later purely via configuration (ADR-0004). Nothing outside this module should import boto3 directly.
`models.py`	—
`scanning.py`	Malware-scan integration for uploaded files (Section 33 of the master prompt). Talks to a ClamAV daemon (`clamd`) over its network socket — never a bundled/vendored scan engine, and never a check that can be satisfied without actually reaching the daemon.
`serializers.py`	—

File	What it is
`services.py`	—
`tasks.py`	—
`urls.py`	—
`views.py`	—

system

File	What it is
`api_urls.py`	—
`backup_inspection.py`	Read-only inspection of a backup set before it is restored (the guided restore's Validate / Preview / Compatibility steps). Never connects to or changes a live database.
`backup_views.py`	The Backups page (Super Administrator only, system.admin): every backup of this installation, whether its restore test passed, and "Back up now". Restoring replaces live data, so it stays in the Control Center on the shop PC (docs/operations/BACKUP_RESTORE.md); nothing here restores.
`backups.py`	Real backup and restore-verification across every store the platform holds data in (Phase 11 added PostgreSQL via `pg_dump` / `pg_restore`; Phase 15 adds object storage and configuration). Restore verification always targets a throwaway, isolated target — a scratch database, a scratch object-storage key prefix — created immediately before and removed immediately after, never production data (Section
`exceptions.py`	—
`health.py`	Health history and alerts. Every 5 minutes (Celery Beat) the checks run and are kept HEALTH_HISTORY_DAYS days; a check failing twice in a row opens an alert, which closes by itself when the check passes again. Besides the dependencies /readyz checks, two things a shop would otherwise notice too late: the last good backup is older than a day and a half, and the backups' disk is nearly full. When no
`management/commands/ensure_tenant_role.py`	—
`management/commands/inspect_backup_set.py`	—
`management/commands/list_backups.py`	—
`management/commands/provision_tenant_role.py`	—

File	What it is
`management/commands/restore_backup.py`	—
`management/commands/restore_backup_file.py`	—
`management/commands/run_backup.py`	—
`management/commands/verify_backup.py`	—
`middleware.py`	—
`models.py`	—
`redaction.py`	Secret redaction for support diagnostics (Local Edition pilot support).
`tasks.py`	—
`tenant_role.py`	Provisions a least-privilege PostgreSQL role for the tenant database connection — CONNECT + CREATE on that one database, never superuser, CREATEDB, or CREATEROLE (docs/security/THREAT_MODEL.md TB3: "the app connects as the container's bootstrap POSTGRES_USER, which is effectively a superuser... isolation between tenant schemas is enforced only by the application layer, not by a second, independent
`throttling.py`	Per-organization request throttling. DRF's built-in throttle classes key by user or by IP; several endpoints (import job creation today) instead need a budget shared across everyone acting within one organization, so that one compromised or careless member of Org A can't exhaust a shared resource without touching Org B's separate budget, and so a single member spinning up many jobs doesn't get a f
`urls.py`	—
`views.py`	—

workspaces

File	What it is
`models.py`	—
`serializers.py`	—
`urls.py`	—
`views.py`	—

Frontend libraries (`apps/frontend/src/lib/`)

File	What it is
`analytics-operations.ts`	Mirrors analytics/data.py's NUMERIC_TYPES/CATEGORICAL_TYPES/TEMPORAL_TYPES exactly — the backend rejects a column of the wrong kind for a given operation with a 400, but filtering the dropdowns to only offer eligible columns up front means an operator only ever discovers that by reading this list once, not by trial and error against the API.
`api.ts`	Talks to the Django/DRF backend at the same origin (Caddy routes /api/v1/* to it — see infrastructure/proxy/Caddyfile), so plain relative fetches with credentials:"include" work with no CORS setup needed, in this deployment or in `next dev` run behind the proxy. Session-cookie auth + Django's CSRF discipline (docs/api/API.md): every mutating request needs an X-CSRFToken header matching the csrftok
`app-pay.ts`	—
`auth-context.tsx`	—
`barcode.ts`	Code 128 (character set B) barcodes for shelf and product labels, drawn as SVG -- no library. Set B covers printable ASCII, which is what item and part numbers use; anything else is refused (the label then shows the text only).
`brand.ts`	The owner's copyright line, shown with the version in the app and in Setup (scripts/shop-kit/setup-stub/Copyright.cs).
`built-in-apps.ts`	The customer application gallery's view of the backend catalog (app_platform/built_in_apps.py). The backend decides what is offered and what its release status is; the UI only presents that honestly. A preview is never dressed up as production-ready.
`business.ts`	Customer-facing business screens over an installed App Platform application.
`environment-type.ts`	—
`format.ts`	Small, dependency-free display helpers for the business screens.
`health.ts`	—
`invoice.ts`	The printed invoice: a complete, self-contained HTML page (every value escaped) for a print window, on A4 paper or on an 80 mm / 58 mm receipt printer. Built from the sale's own records (SalesDocument.tsx); it changes nothing.
`label-image.ts`	Labels as PNG pictures, drawn on a canvas at 300 dpi with the same layout as the printed labels (Labels.tsx): the shop name, the product name, its details, the barcode, the captioned number, price and shelf. For a label printer's own software, a print shop, or sending by phone.
`license.ts`	This installation's licence (ADR-0023): a 30-day trial, then read-only until a serial number is activated for this PC with the activation code the vendor sends back.
`medicine-import.ts`	Importing a medicines catalogue (a CSV such as netmeds' medicines.csv: disease_name, med_name, med_url, final_price, price, prescription_required, drug_varient, drug_manufacturer, drug_manufacturer_origin, drug_content, generic_name, img_urls) into the Pharmacy.

File	What it is
`movies.ts`	Movie Shop (ADR-0022): types and calls for the owner's catalogue. The films stay on the shop's drives; IntraForge keeps their names, sizes, covers and prices.
`org-resources.ts`	—
`passkeys.ts`	Passkeys (ADR-0019) with the browser's own WebAuthn API -- no library. The server sends the options as JSON (binary fields base64url-encoded) and checks the answer; this file only converts between JSON and the ArrayBuffers `navigator.credentials` wants, and back.
`portal.ts`	The Movie Shop customer portal (ADR-0022 phase 3), at /portal/<shop>. Customers are not IntraForge users: the portal API takes only its own HttpOnly cookie (set by sign-up / sign-in), and every change carries X-Portal-Request, which another site cannot send. Nothing here uses IntraForge's session or CSRF token.
`project-trail.ts`	—
`queries.ts`	—
`query.tsx`	—
`runtime-names.ts`	—
`safe-next.ts`	Validates a `?next=` return-to path before ever navigating to it. Must be a same-origin, absolute-from-root path -- never a scheme- relative ("//evil.example.com"), absolute ("https://evil.example.com"), or backslash-prefixed ("\\evil.example.com", which some browsers still treat as scheme-relative) URL. This is the one thing standing between "/login?next=..." and being an open redirect.
`service-apps.ts`	—
`shop-details.ts`	The shop's own details for what IntraForge prints: its name on every label, and its name, address, phone, tax number, currency and closing note on every invoice. Kept per business app on this browser (each counter PC fills them in once); nothing here is sensitive, and the app's records stay the only source of what was sold.
`sites.ts`	Hosted websites and Node.js apps (ADR-0019 / ADR-0020), served by the proxy on their own port.
`test-schema.ts`	A miniature Spare Parts schema for component tests: ids are opaque and resolved from keys, exactly as `loadSchema` does from an instance's own metadata. Test support only -- never imported by the app.
`theme.ts`	Light, dark or the computer's own choice (globals.css: dark mode redefines the colour variables under data-theme="dark" on <html>). The choice is kept in this browser only.
`tour.ts`	When the welcome tour shows: once per person on this browser, the first time they reach the app. Setup's last screen opens IntraForge with `?tour=1` ("Show me a quick tour" ticked) or `?tour=off` (unticked); that wish is kept on this browser until the person is signed in, because the address they land on is the sign-in or registration page, not the app.
`use-dialog-a11y.ts`	—

File	What it is
`use-reduced-motion.ts`	—

Frontend components (`apps/frontend/src/components/`)

File	What it is
`AnalyticsResult.tsx`	—
`AppModelAccess.tsx`	—
`AppShell.tsx`	—
`business/BusinessApp.tsx`	—
`business/hooks.ts`	—
`business/Inventory.tsx`	—
`business/InvoicePrint.tsx`	—
`business/Labels.tsx`	—
`business/MedicineImport.tsx`	—
`business/Overview.tsx`	—
`business/PartNumbers.tsx`	—
`business/ProductDetail.tsx`	—
`business/ProductPicker.tsx`	—
`business/ReceiveStock.tsx`	—
`business/RecordSection.tsx`	—
`business/Sales.tsx`	—
`business/SalesDocument.tsx`	—
`business/Sections.tsx`	—
`business/SellStock.tsx`	—

File	What it is
`business/ToyoDiyLookup.tsx`	—
`CommandPalette.tsx`	—
`ConfirmProvider.tsx`	—
`DeveloperNav.tsx`	—
`DeveloperStub.tsx`	—
`home/DemoScenes.tsx`	The four scenes of the home page's product demo (ProductDemo.tsx). Drawn on a fixed 640 x 420 stage that the demo
`home/ProductDemo.tsx`	—
`home/Reveal.tsx`	—
`home/SetupWalkthrough.tsx`	—
`LicenceBanner.tsx`	—
`movies/AdBanner.tsx`	—
`movies/Ads.tsx`	—
`movies/Catalogue.tsx`	—
`movies/Checkout.tsx`	—
`movies/Customers.tsx`	—
`movies/Dialogs.tsx`	—
`movies/MovieCover.tsx`	—
`movies/MovieShops.tsx`	—
`movies/SalesList.tsx`	—
`PageTransition.tsx`	—
`PasswordForm.tsx`	—
`portal/AdCarousel.tsx`	—
`portal/AuthScreen.tsx`	—
`portal/Browse.tsx`	—
`portal/CartOrders.tsx`	—
`portal/MeScreen.tsx`	—
`portal/Portal.tsx`	—
`RecoveryCodes.tsx`	—
`ShareSection.tsx`	—
`ThemeToggle.tsx`	—
`tour/TourArt.tsx`	The welcome tour's five drawings (design: design-references/installer-mockups/tour.js). Decorative only: every fact

File	What it is
`TourRequestCapture.tsx`	—
`ui.tsx`	—
`WelcomeTour.tsx`	—

The shop kit (`scripts/shop-kit/`)

File	What it is
`build-setup-exe.sh`	Wraps a shop-kit zip (and, optionally, the Windows prerequisites) into ONE double-click self-extracting installer.
`build-shop-kit.sh`	Builds the IntraForge Local Edition own-shop pilot kit -- run ONCE, ONLINE, on the build machine (Docker Desktop running).
`build-spareparts-image.sh`	Builds the Spare Parts service-application image (ADR-0018) from a COMMITTED revision of the Spare Parts project: `git archive` of REF, so no working-tree file (.env*, local data, node_modules, build output) can reach the image.
`control-center/ControlCenter.cs`	IntraForge Control Center (shop kit, Docker Desktop runtime).
`control-center/Ui.cs`	The Control Center's look and feel: one visual language for every window (DESIGN.md -- navy brand, blue accent, Segoe UI): a dark sidebar, white cards on a light canvas, clear status colours and icon glyphs from the icon font Windows itself ships (Segoe Fluent Icons on Windows 11, Segoe MDL2 Assets on Windows 10), so nothing is installed.
`media-agent/build-media-agent.sh`	Compiles the Movie Shop media agent (ADR-0022) with the C# compiler that ships with Windows (.NET Framework 4.x). bash scripts/shop-kit/media-agent/build-media-agent.sh <output.exe
`media-agent/MediaAgent.cs`	IntraForge media agent (ADR-0022) -- the Movie Shop's hands on the shop PC. C# 5 / .NET Framework 4.x, built with the compiler that ships with Windows (build-media-agent.sh). It runs as the signed-in user, so it sees the same drives the owner sees (internal disks, external disks, USB sticks, NAS shares mapped as a drive letter).
`setup-stub/build-stub.sh`	Compiles the IntraForge Setup window (this folder) into one exe with the C# compiler that ships with Windows (.NET Framework 4.x): no SDK needed to build it, nothing needed on the PC that runs it. The icon, the manifest and the two brand images are compiled in. SETUP_VERSION=1.5.2-abc bash scripts/shop-kit/setup-stub/build-stub.sh <output.exe
`setup-stub/Copyright.cs`	The owner's copyright, shown in the Setup window and in the exe's Windows file properties (Details tab).
`setup-stub/Log.cs`	Where Setup's messages go: the console (the /check, /quiet and /extractonly modes) or the Setup window's details log.
`setup-stub/Payload.cs`	The packages carried inside IntraForge-Setup.exe: reading the trailer, checking the SHA-256 of each part, and unpacking. [stub][kit .zip][Docker Desktop installer][WSL package .msi][SHA-256 x3][length x3][magic "IFSFX003"] (128-byte trailer; a length of 0 means "not included").

File	What it is
`setup-stub/PcCheck.cs`	Whether this PC can run IntraForge and Docker Desktop: Windows version, memory, disk, virtualization, Docker.
`setup-stub/Program.cs`	IntraForge Setup -- a self-extracting installer (C# 5, .NET Framework 4.x: present on every Windows 10/11 PC).
`setup-stub/Ui/Icons.cs`	Line icons in the style of lucide.dev (the web app's icon set), drawn on a 24-unit grid with GDI+.
`setup-stub/Ui/InstallRun.cs`	Runs an install or upgrade on a background thread and turns what the kit's scripts print into the window's progress: which step is running, how far the whole thing has got, and the full log for "Show details".
`setup-stub/Ui/Pages.cs`	What each Setup screen looks like (design: design-references/installer-mockups). Logical pixels, 880 x 560.
`setup-stub/Ui/Preview.cs`	/uipreview=<screen>: shows one Setup screen with sample data, for screenshots and documentation. Installs nothing.
`setup-stub/Ui/SetupForm.cs`	The Setup window: a borderless 880 x 560 window (Windows 11 rounds its corners and draws its shadow) with a brand panel on the left and one screen at a time on the right. Every pixel is drawn here (Pages.cs); only the two folder boxes and the details log are real Windows controls.
`setup-stub/Ui/Slides.cs`	"While you wait": the five slides the brand panel shows during the install -- what IntraForge does, one idea each.
`setup-stub/Ui/Theme.cs`	The Setup window's look: DESIGN.md colour tokens, Segoe UI type, and the drawing helpers every screen uses. All sizes are in logical pixels on an 880 x 560 window; the window scales the whole drawing to the screen's DPI.
`template/Backup-IntraForgeShop.ps1`	Takes a full backup of the shop (company data, business records and stored files) into the backups folder, and records whether it worked.
`template/Install-IntraForgeShop.ps1`	Installs IntraForge Local Edition on this shop PC with NO internet: checks the PC, loads the images from this kit, creates fresh secret keys, starts everything, and sets up automatic start, a daily backup and desktop shortcuts.
`template/Install-Prerequisites.ps1`	Step B of preparing a clean Windows PC (needs administrator rights; run by Setup-IntraForge.ps1, one UAC prompt): installs the WSL package and Docker Desktop from the copies in prerequisites\ (no download). Each installer is run only if it carries a valid signature from its publisher (Microsoft Corporation / Docker Inc).
`template/IntraForgeShop.Common.ps1`	Shared helpers for the IntraForge own-shop pilot scripts. Dot-sourced by the other scripts; never run on its own. Written for Windows PowerShell 5.1 (what a stock Windows 10/11 PC has).

File	What it is
`template/Prepare-WindowsFeatures.ps1`	Step A of preparing a clean Windows PC (needs administrator rights; run by Setup-IntraForge.ps1, one UAC prompt): switches on the Windows parts Docker Desktop's WSL 2 engine needs, and makes the installing user a member of docker-users so the NEXT sign-in can talk to Docker.
`template/Repair-IntraForgeBackups.ps1`	Repairs "pg_dump: ... Permission denied" backups: gives the Windows user who runs Docker Desktop write access to the IntraForge backups and logs folders, checks that IntraForge can now write there, and takes one backup.
`template/Restore-IntraForgeShop.ps1`	Puts the shop back to the state of a saved backup. DESTRUCTIVE: everything entered after that backup is lost.
`template/Set-IntraForgeNetwork.ps1`	Lets the other PCs in this shop open IntraForge, or makes it this-computer-only again (the default).
`template/Setup-IntraForge.ps1`	Sets up IntraForge on this PC from start to finish, including everything a clean Windows needs first, with no internet: 1. checks the PC (virtualization, ...); 2. switches on the Windows parts Docker's WSL 2 engine needs (one administrator prompt) -> RESTART; 3. installs the WSL package and Docker Desktop from prerequisites\ (one administrator prompt); 4. installs IntraForge (Install-IntraForgeSho
`template/Start-IntraForgeShop.ps1`	Starts IntraForge (if it is not already running) and opens it in the browser. Runs by itself at Windows sign-in.
`template/Status-IntraForgeShop.ps1`	Shows whether IntraForge is healthy and saves the answer to a file the project team can read. Contains no passwords or keys (only their presence).
`template/Stop-IntraForgeShop.ps1`	Stops IntraForge. All data is kept. It starts again at the next sign-in or with "Open IntraForge".
`template/Uninstall-IntraForgeShop.ps1`	Removes IntraForge from this PC in one of three ways (the Control Center's Uninstall and Windows' Installed apps lead here):
`template/Upgrade-IntraForgeShop.ps1`	Upgrades an installed IntraForge to this kit's version, keeping all data. Run by the setup file when it finds an earlier version installed (it unpacks the new kit next to the old one and starts this script from there).
`vm-test/Prepare-VmTest.ps1`	Hyper-V side of testing IntraForge-Setup-Complete.exe inside the Windows 11 test VM. Run it in an ADMINISTRATOR PowerShell (Hyper-V management needs it). It never touches anything inside the VM except the one file it copies in.